

Declaration

I declare the following:

(1) that the material contained in this dissertation is the end result of my own work and that due acknowledgement has been given in the bibliography and references to **ALL** sources be they printed, electronic or personal.

(2) the Word Count of this Dissertation is 14228.

(3) that unless this dissertation has been confirmed as confidential, I agree to an entire electronic copy or sections of the dissertation to being placed on the eLearning Portal, if deemed appropriate, to allow future students the opportunity to see examples of past dissertations. I understand that if displayed on the eLearning Portal it would be made available for no longer than five years and that students would be able to print off copies or download.

(4) I agree to my dissertation being submitted to a plagiarism detection service, where it will be stored in a database and compared against work submitted from this or any other Department or from other institutions using the service.

In the event of the service detecting a high degree of similarity between content within the service this will be reported back to my supervisor and second marker, who may decide to undertake further investigation that may ultimately lead to disciplinary actions, should instances of plagiarism be detected.

(5) I have read the University of Sunderland Policy Statement on Ethics in Research and I confirm that ethical issues have been considered, evaluated and appropriately addressed in this research.

SIGNED: Adewale James Ilesanmi

DATE: August 12, 2024

Abstract

In today's competitive business environment, understanding customer behavior and preferences is crucial for companies to develop effective marketing strategies and maintain a competitive edge. Customer segmentation, which involves dividing customers into distinct groups based on demographic, behavioral, and transactional characteristics, is essential for refining marketing approaches and enhancing customer satisfaction. This research evaluates the effectiveness of base autoencoders and VAE (VAEs) in identifying meaningful customer segments. The quality of clustering was assessed using silhouette scores and the Calinski-Harabasz index.

Initial experiments without dimensionality reduction showed that autoencoders provided moderate clustering performance, while VAEs were less effective. To improve results, PCA (Principal Component Analysis) was incorporated, leading to significantly enhanced clustering for the autoencoder. The autoencoder with PCA outperformed traditional clustering methods such as K-means and DBSCAN, which also benefited from the integration of PCA. Although the VAE with PCA showed some improvement, it remained less effective than the autoencoder with PCA.

These findings highlight the potential of autoencoder-based customer segmentation to drive targeted marketing strategies and boost business competitiveness. This approach offers a valuable tool for companies seeking to gain a deeper understanding of their customer base. Future research could explore the application of this method across various industries and assess its effectiveness in real-world marketing scenarios.

Contents

DECLARATION.....	1
ABSTRACT.....	2
1 INTRODUCTION.....	7
1.1 BACKGROUND.....	7
1.2 AIMS	8
1.3 OBJECTIVES	9
1.4 RESEARCH APPROACH	9
1.5 STRUCTURE OF THE REPORT	10
1.6 ETHICAL, SOCIAL, PROFESSIONAL, LEGAL AND SECURITY CONSIDERATIONS	10
1.6.1 Professional Considerations.....	10
1.6.2 Ethical Considerations	11
1.6.3 Legal Considerations	11
1.6.4 Social Considerations.....	12
2 LITERATURE REVIEW ON AUTOENCODER FOR SEGMENTATION	13
2.1 REVIEW ON CUSTOMER SEGMENTATION AND PRACTISES	13
2.2 MACHINE LEARNING IN CUSTOMER SEGMENTATION	15
2.2.1. Review of existing studies	16
2.3 DEEP LEARNING MODELS	18
2.1 Autoencoders	20
2.2 Hyperparameters in autoencoder	23
2.4. SUMMARY OF FACTORS FOR EFFECTIVE AUTOENCODER MODEL	24
2.5 RESEARCH GAP.....	25
3 PRACTICAL RESEARCH METHODOLOGY	26
3.1 INTRODUCTION	26
3.2 DATA UNDERSTANDING.....	27
3.3 DATA PREPARATION.....	29

3.4	MODELLING	31
3.4.1	Autoencoder Base model Architecture	32
3.4.2	Variational Autoencoder model Architecture	33
3.4.3	Model Training	34
3.4.4	Hyperparameter Tuning of the Autoencoders.....	35
3.5	SUMMARY OF EXPERIMENT CONDUCTED	36
3.6	SEGMENTATION	37
3.7	EVALUATION	38
	Justification for Metric selection	40
4	ANALYSIS OF RESULTS.....	42
4.1	EXPERIMENT ONE MODEL RESULTS (WITHOUT PCA)	42
4.1.2	Results.....	44
4.2	EXPERIMENT TWO MODEL RESULTS (WITH PCA).....	46
4.2.2	Results.....	46
4.3	EXPERIMENT THREE MODEL RESULTS (TRADITIONAL METHODS).....	48
4.3.1	Results.....	49
4.4	COMPARISON OF RESULT OF THE EXPERIMENT AND WITH EXISTING LITERATURE	50
	Existing Literature report	52
5	PROJECT EVALUATION AND REFLECTION.....	54
5.1	OVERVIEW OF PROJECT.....	54
5.2	OBJECTIVES REVIEW.....	54
5.3	EVALUATION OF METHODOLOGY AND RESULTS.....	56
5.4	PERSONAL REFLECTION	58
5.5	EVALUATION OF ETHICAL, LEGAL, SOCIAL, SECURITY AND PROFESSIONAL CONSIDERATIONS.....	59
5.6	LIMITATIONS OF THE STUDY	61
6	CONCLUSION AND RECOMMENDATIONS.....	62
6.1	CONCLUSION.....	62

6.2	RECOMMENDATIONS.....	63
6.2.4	Inclusivity and Accessibility.....	63
6.2.5	Practical Applications	63
6.2.6	Further Research	64
6.2.7	Alternative Approaches	64
7	REFERENCE LIST.....	65
	APPENDIX A: CODE	77
	APPENDIX B: RESEARCH PROPOSAL	93

LIST OF TABLES

Table 1	Data summary and description	27
Table 2	Result based on the segmentation without PCA.....	44
Table 3	Result based on the segmentation with PCA	46
Table 4	Performance of clustering using Traditional method	49
Table 5	combined performances across each method.....	51

LIST OF FIGURES

Figure 1	Common Criteria or Characteristics Used for Customer Segmentation	14
Figure 2	Classification of deep learning techniques	19
Figure 3	Architecture of autoencoder model	21
Figure 4	CRISP-DM for Data Science (Schröer et al., 2021).....	26
Figure 5	section of preprocessing	30
Figure 6	Model Architecture for Autoencoder	33
Figure 7	Model Architecture for VAE.....	34
Figure 8	Summary of implementation	37
Figure 9	Training and validation loss over Epochs for the base autoencoder model	42

Figure 10 Training and validation loss over Epochs for the base VAE model.....	43
Figure 11 Autoencoder Clusters	45
Figure 12 Variational Autoencoder Clusters	45
Figure 13 Autoencoder PCA Clusters.....	47
Figure 14 Variational Autoencoder PCA Clusters.....	47
Figure 15 Experiment and model performance against evaluation metrics.....	52
Figure 16 Clustering performance for existing result	53

DEFINITION OF NOTATIONS

Notation	Description
AE	Autoencoder
VAE	Variational Autoencoder
PCA	Principal component Analysis

1 Introduction

1.1 Background

In today's competitive business environment, understanding customer behaviour and preferences is crucial for companies to develop effective marketing strategies and maintain a competitive edge. Customer segmentation, a process of dividing customers into distinct groups based on their demographic, behavioural, and transactional characteristics (Hassan, 2018). This is a crucial aspect of marketing strategy as it involves dividing a customer base into distinct groups based on their shared characteristics, behaviours, and preferences (Yoseph et al., 2020). Effective customer segmentation allows businesses to better understand their target audience (An et al., 2018), tailor their products and services to meet specific needs (Christy et al., 2018) and optimize their marketing efforts for maximum impact and cost-effectiveness (Zhou, Zhai and Pantelous, 2020). Conventional segmentation methods, such as demographic, geographic, psychographic, and behavioural segmentation, have been widely employed by organizations (Nica, Gajanova, and Kicova, 2019). However, with the advent of big data and advanced analytics, there is a growing need for more sophisticated and data-driven approaches to customer segmentation.

Nowadays machine learning gained popularity as a tool for customer segmentation. Machine learning can do an efficient analysis of large data sets (Nagaraj et al., 2023). In such kind of data set identifying the patterns would be difficult manually, but machine learning can identify patterns more efficiently and accurately. Unsupervised machine learning techniques, particularly clustering algorithms, have emerged as powerful tools for identifying patterns and grouping customers based on their similarities (Ezugwu et al., 2022). These algorithms can process large amounts of data and uncover hidden relationships or structures within the data without relying on pre-defined labels or categories. These methods, including k-means and hierarchical clustering, group customers into clusters without predefined labels, making them suitable for exploratory data analysis where the goal is to discover natural groupings within the data (Fuchs and Wolfram Höpken, 2022). However, while these traditional clustering methods are effective, they have limitations in capturing complex, non-linear relationships in the data.

Among the various unsupervised learning techniques, autoencoders have gained significant attention in recent years due to their ability to learn efficient data representations and capture complex patterns in high-dimensional data (Chen and Guo, 2023).

Autoencoders, a type of artificial neural network used for unsupervised learning, have shown great promise in overcoming these limitations. Through learning of compressed representation of the data (encoding) and then reconstructing the original data (decoding), autoencoders can capture more intricate patterns and structures (Pawar and Attar, 2019). This makes them particularly suitable for tasks like anomaly detection, dimensionality reduction, and, importantly, customer segmentation. Through this ability, autoencoders could learn complex representations enabling businesses to potentially identify more accurate and meaningful customer segments (Yang, Zhang, and Kannan, 2021). This can lead to more effective targeted marketing strategies, as the segmentation is based on deeper insights into customer behaviours and preferences (Griva et al., 2018). Despite the growing interest in using autoencoders for customer segmentation, there is a need for further research to explore the effectiveness of this approach and its potential advantages over traditional clustering methods. This dissertation investigates the use of autoencoders for predicting customer segments, comparing their performance with traditional clustering methods. Analysing the strengths and weaknesses of each approach, this study aims to contribute to the growing body of knowledge in machine learning applications in marketing and provide practical insights for businesses looking to enhance their customer segmentation strategies.

1.2 Aims

The primary aim of this project is to explore the application of unsupervised machine learning, specifically autoencoders, to predict customer segments based on demographic and behavioural characteristics. The study seeks to assess the effectiveness of autoencoders over traditional clustering methods in identifying distinct customer groups, thereby offering insights for more precise and effective targeted marketing strategies.

1.3 Objectives

1. To identify key factors for effectively building an autoencoder model by conducting a comprehensive literature review of peer-reviewed articles published within the last five years, completing the review within two months to improve clustering performance using autoencoder models.
2. To Investigate and compare the performance of AE, VAE and classic clustering algorithms in customer segmentation models using silhouette scores and the Calinski-Harabasz index as performance metrics, through training and testing these models with demographic and customer behavioural data.
3. To assess the impact of PCA as an additional dimension reduction technique on clustering results by analyzing the performance change using silhouette scores and Calinski-Harabasz indices before and after applying PCA, to determine the effectiveness of PCA in improving clustering performance.
4. To investigate and provide actionable recommendations on the comparison of VAE, and traditional clustering algorithms for customer segmentation practices, through detailed reports on performance metrics and practical insights to enhance customer segmentation strategies in businesses.
5. To follow the project schedule detailed in Appendix B, making sure all goals are met within the set deadlines. The schedule outlines key milestones and includes biweekly progress reviews to monitor compliance and make adjustments as needed, ensuring the project remains on course for on-time completion.

1.4 Research Approach

This study will employ a mixed-methods approach, combining both qualitative and quantitative methods to achieve its objectives. The research will begin with a comprehensive literature review to analyse existing research on unsupervised machine learning techniques for customer segmentation, with a focus on autoencoder methods. This is followed by collection and preprocessing of demographic and behavioural data of customers. The data will be split into training and testing sets, with the training set used to develop and train the autoencoder model, and the testing set used to evaluate its performance. The autoencoder model will be designed,

trained, and tested using Python and the Keras library, in identifying distinct customer segments. The performance of the autoencoder model will be evaluated using metrics such as silhouette score, Calinski-Harabasz index, and clustering accuracy. Additionally, the study will compare the performance of the autoencoder model with traditional clustering algorithms (e.g., k-means, hierarchical clustering) using the same dataset, and quantify the improvement in identifying distinct customer segments.

1.5 Structure of the Report

This report will be structured into six chapters, including an introduction, literature review, methodology, results, discussion, and conclusion. The introduction will provide an overview of the research, including the background, aims, objectives, and research approach. The literature review will analyse existing research on unsupervised machine learning techniques for customer segmentation, with a focus on autoencoder methods. The methodology chapter will describe the research design, data collection and preprocessing, and the development and testing of the autoencoder model. The results chapter will present the findings of the study, including the performance of the autoencoder model and its comparison with traditional clustering algorithms. The discussion chapter will interpret the results, discuss the implications of the study, and identify areas for future research. Finally, the conclusion chapter will summarize the main findings of the study and provide recommendations for businesses on how to apply autoencoders for targeted marketing strategies.

1.6 Ethical, Social, Professional, Legal and Security Considerations

1.6.1 Professional Considerations

Primarily, it is essential to adhere to the principles of academic integrity throughout a research process (Davies, 2018). This includes accurately citing and referencing all sources used in the literature review, avoiding plagiarism, and maintaining transparency in the reporting of methods, data, and findings. Fabrication or manipulation of data, as well as selective reporting of results, would be a breach of academic integrity and will strictly be avoided. Furthermore,

the research will be conducted with objectivity and impartiality, without bias or preconceived notions influencing the analysis or interpretation of results. It is essential to maintain a critical and analytical approach throughout the research process, acknowledging potential limitations or sources of error, and presenting findings in a balanced and objective manner will be considered in this study.

1.6.2 Ethical Considerations

A key ethical concern is data privacy and the protection of individuals' personal information. Customer datasets often contain sensitive information, such as demographic details, purchase histories, and behavioural patterns. It is essential to ensure that this data is handled with utmost care and confidentiality, adhering to relevant data protection regulations and industry best practices. Measures such as data anonymization, using open-source dataset, secure storage, and restricted access will be implemented to safeguard individual privacy and prevent unauthorized access or misuse of data if non-open-source data will be used. Furthermore, the transparency and interpretability of the autoencoder model will be considered. As complex neural networks can often be perceived as "black boxes," it is important to strive for explainability and interpretability of the model's decisions and outputs (Chakraborty, 2024). This transparency not only supports ethical decision-making but also fosters trust and acceptance among stakeholders, including customers and businesses.

1.6.3 Legal Considerations

One of the primary legal considerations is data protection and privacy regulations. Depending on the jurisdiction and the nature of the customer data being used (for this study open-source data is considered), various laws and regulations may apply, such as the General Data Protection Regulation (GDPR) in the European Union or the Data Protection Act in the United Kingdom. Furthermore, the research project will comply with industry-specific regulations or standards, particularly if the customer data or the application of the autoencoder model is related to sensitive sectors such as finance, healthcare, or telecommunications.

1.6.4 Social Considerations

The application of machine learning techniques, such as autoencoders, for customer segmentation has the potential to impact society in many ways. One of which is the potential for algorithmic bias and discrimination. As mentioned in the ethical considerations, machine learning algorithms can inadvertently perpetuate or amplify existing biases present in the training data, leading to discriminatory outcomes. In the context of customer segmentation, biased algorithms could result in certain customer groups being treated unfairly or subjected to unfavourable marketing practices based on their demographic or behavioural characteristics. So, model and data will be explored to identify any known bias within the dataset. Another social consideration is the potential impact on consumer privacy and autonomy. While targeted marketing strategies can be beneficial for businesses and customers alike, there is a risk of overly intrusive or manipulative practices that infringe on individual privacy and personal autonomy. As such a balance will be struck between effective marketing practices, algorithm selections, implementation and respecting individual privacy and choice.

2 Literature Review on Autoencoder for segmentation

2.1 Review on Customer segmentation and practises

Over the years, as there is extraordinarily strong competition in the business world, the organizations must enhance their profits and business by satisfying the demands of their customers and attract new customers according to their needs (Camilleri, 2019). The identification of customers and satisfying the demands of each customer is a complex and tedious task (Alkhayrat et al., 2020). This is because customers may be different according to their demands, tastes, preferences and so on. Instead of "one-size-fits-all" approach, customer segmentation clusters the customers into groups sharing the same properties or behavioural characteristics (Camilleri, 2019).

Customer segmentation (also known as market segmentation) is a marketing practice of dividing a customer base into sub-groups as described by Nagaraj et al., (2023). The key to effective segmentation is to divide customers into groups based on, the prediction of their value to the business. After that target, each group with different strategies to extract maximum value from both high and low-profit customers (Varad, Thalkar, 2021). The primary goal of customer segmentation is to identify and target specific customer groups that are likely to respond similarly to a marketing strategy or have similar needs (Shinde, 2023).

Traditionally, customer segmentation has relied on demographic factors (age, gender, income), geographic location, psychographic attributes (lifestyle, values), and behavioural patterns (purchasing habits, brand loyalty) as shown in figure 1 (Rakeshkumar et al., 2023). These methods often involve statistical techniques such as cluster analysis, factor analysis, and discriminant analysis (Gomes and Meisen, 2023). Several theories support customer segmentation, including customer behaviour theory and market segmentation theory. Customer behaviour theory suggests that customers' purchasing decisions are influenced by their attitudes, perceptions, and motivations (Varad R Thalkar, 2021). Market segmentation theory

posits that a market can be divided into distinct segments based on customers' needs, preferences, and behaviours (Manuere et al., 2022).



Figure 1 Common Criteria or Characteristics Used for Customer Segmentation

Machine learning has revolutionized customer segmentation by enabling businesses to analyse large datasets and identify complex patterns and relationships that may not be apparent through traditional methods (Sakshi et al., 2023). According to Alam et al., (2021), Machine learning algorithms, such as clustering and decision trees, can be used to segment customers based on their behaviour, preferences, and characteristics. Additionally, machine learning can help businesses identify new customer segments and adapt to changing customer needs and preferences over time (Alkhayrat et al., 2020). In recent years, there has been a growing interest in using unsupervised machine learning methods, such as Autoencoders, for customer segmentation (Ryu et al., 2020). Autoencoders are designed to learn efficient coding of input data, often for the purpose of dimensionality reduction or feature learning (Pumsirirat and Yan, 2018). By compressing data into a lower-dimensional representation and then reconstructing it, autoencoders can uncover intricate patterns and relationships within the data. This makes

them particularly suited for segmenting customers in ways that capture deeper behavioural insights than traditional methods (Tavakoli et al., 2020). As businesses accumulate more data and seek to personalize customer experiences in real-time, machine learning approaches to segmentation are becoming increasingly valuable. They offer the potential for more accurate, dynamic, and actionable customer insights, enabling businesses to adapt quickly to changing market conditions and customer needs (Tavakoli et al., 2020).

2.2 Machine learning in customer segmentation

Machine learning has revolutionized the field of customer segmentation by enabling businesses to analyse large datasets and identify complex patterns and relationships that may not be apparent through traditional methods (Monil, 2020). Machine learning algorithms can be broadly classified into two categories: supervised and unsupervised methods.

Supervised methods, such as decision trees, random forests, and support vector machines, as pointed out by Ozan, (2018) rely on labelled data to learn patterns and make predictions about customer segments. These methods excel in scenarios where clear, predefined segments exist and can be used to classify new customers into these known categories (Ghosal et al., 2019). The researchers stated that they offer high accuracy and interpretability, making them valuable for businesses with well-established customer bases and clearly defined market segments. However, supervised methods are limited by their reliance on labelled data, which can be expensive and time-consuming to obtain, and may not capture emerging or previously unknown customer segments (Shen, 2021).

Unsupervised methods, on the other hand, do not require labelled data and are designed to discover inherent structures within the data (Patankar et al., 2021). These techniques, including k-means clustering, hierarchical clustering, and more advanced approaches such as autoencoders, have gained significant popularity in customer segmentation due to their ability to identify natural groupings without preconceived notions of segment boundaries (Turkmen, 2022). Unsupervised methods are particularly valuable in dynamic markets where customer behaviour is constantly evolving, as they can uncover new and unexpected segments that traditional approaches might miss (Yuping et al., 2020).

2.2.1. Review of existing studies

K-means clustering, one of the most widely used unsupervised methods, has been applied extensively in customer segmentation due to its simplicity and efficiency. For example, a study by Kumar, (2023) used k-means clustering to segment customers in the banking industry based on their transaction patterns and demographics, revealing distinct groups with different banking behaviours and needs. Similarly, Sarvari et al., (2016) applied k-means clustering to segment customers of a hair salon, identifying groups based on recency, frequency, and monetary value of their purchases.

Hierarchical clustering, another popular unsupervised method, has been employed in various customer segmentation studies. Randa Mokhtar (2024), used hierarchical clustering in combination with association rules to segment customers of an online retailer, providing insights into purchasing patterns and preferences of different customer groups. This approach allowed for a better understanding of customer behaviour compared to traditional RFM (Recency, Frequency, Monetary) analysis (Zhao, 2024).

More advanced unsupervised techniques, such as self-organizing maps (SOMs) and fuzzy clustering, have also been explored in customer segmentation research. Gözde ASLANTAŞ et al., (2023) used SOMs to segment customers in the telecommunications industry, demonstrating the method's ability to handle high-dimensional data and visualize complex customer relationships. Fuzzy clustering, which allows customers to belong to multiple segments with varying degrees of membership, has been applied in studies like that of Chugh, (2021) who used it to segment hotel customers based on their satisfaction levels across different service attributes. Despite the widespread use and success of these traditional unsupervised methods, they are not without limitations. K-means clustering, while popular and computationally efficient, assumes spherical cluster shapes and struggles with clusters of varying sizes and densities as emphasized by Zare and Emadi, (2020) who stated that it also requires the number of clusters to be specified in advance, which can be challenging when the optimal number of customer segments is unknown. Hierarchical clustering, while not requiring a predefined number of clusters, can be computationally intensive for large datasets and is sensitive to outliers (Ikotun et al., 2022).

These limitations have led researchers to explore more unsupervised learning approaches, such as density-based clustering algorithms like DBSCAN (Density-Based Spatial Clustering of Applications with Noise) and OPTICS (Ordering Points to Identify the Clustering Structure) (Umuhoza et al., 2020). These methods can identify clusters of arbitrary shapes and handle noise in the data more effectively. For instance, Deng, (2020) demonstrated the application of DBSCAN in customer segmentation, showing its ability to identify clusters of varying densities and shapes, which is particularly useful in markets with diverse customer behaviours. The need for more approaches has also driven interest in probabilistic models like Gaussian Mixture Models (GMMs) and Latent Dirichlet Allocation (LDA) (Mohamad and Abdelhamid Bouchachia, 2020). GMMs offer a flexible approach to modelling complex, multi-modal distributions in customer data, allowing for overlapping segments and providing probabilistic assignments of customers to segments (Olamendy, 2024). LDA, originally developed for topic modelling in text analysis, has been adapted for customer segmentation by treating customers as documents and their behaviours or attributes as words (Apgar, 2023). This approach can uncover latent patterns in customer behaviour that may not be apparent through traditional clustering methods (Abdar et al., 2021).

The application of machine learning in customer segmentation extends beyond just identifying segments it also encompasses predicting customer behaviour, personalizing marketing efforts, and optimizing customer lifetime value (Smolic, 2024). Reinforcement learning techniques have been explored for dynamic segmentation, adapting to changing customer behaviours in real time (Rocca, 2020). Ensemble methods, combining multiple machine learning algorithms, have shown promise in improving the robustness and accuracy of segmentation models (Basu et al., 2023; Kingma and Welling, (2019)). While machine learning approaches offer significant advantages in terms of handling high-dimensional data and uncovering patterns, they also present challenges. Interpretability remains a key issue, as businesses need to understand and explain the basis for segmentation decisions (Schlager and Christen, 2021). Efforts to develop more interpretable models, such as attention mechanisms in neural networks, are ongoing. Data quality and privacy concerns also pose challenges, especially with the increasing focus on data protection regulations like GDPR (Saeed and Omlin, 2023).

The future of machine learning in customer segmentation lies in developing more adaptive, context-aware models that can integrate multiple data sources and adapt to rapidly changing consumer behaviours (Zahra et al., 2024). Transfer learning techniques, which allow models trained on one domain to be applied to related domains, show promise in addressing the challenge of limited data in new markets or product lines (Chen et al., 2020). Federated learning approaches are being explored to enable collaborative model training across multiple organizations while preserving data privacy. As the field continues to evolve, the integration of machine learning with domain expertise and traditional marketing theories remains crucial. While advanced algorithms can uncover complex patterns, the interpretation and application of these insights require a deep understanding of business contexts and customer psychology. Hybrid approaches that combine the strengths of machine learning with human expertise are likely to yield the most effective customer segmentation strategies.

In recent years, deep learning techniques have emerged as powerful tools for customer segmentation, addressing many limitations of traditional methods. Autoencoders have shown promise in handling high-dimensional and customer data (Sinaga and Yang, 2020). These neural network architectures learn to compress the input data into a lower-dimensional representation and then reconstruct it, effectively capturing the most salient features for segmentation (Shams et al., 2023). Li et al. (2019) demonstrated the use of autoencoders for customer segmentation in the retail industry, showing their ability to handle diverse data types and uncover non-linear relationships in customer behaviour.

2.3 Deep learning models

Deep learning, a subset of machine learning, has revolutionized various fields including customer segmentation (Sarker, 2021). Deep learning models are artificial neural networks with multiple layers that can learn hierarchical representations of data (Md. Eshmam et al., 2024). In customer segmentation, deep learning models offer the potential to uncover customer behaviours and preferences that traditional methods might miss as stated by Li et al., (2023). Deep learning models like traditional ML could be group in supervised and unsupervised methods as shown in figure 2.

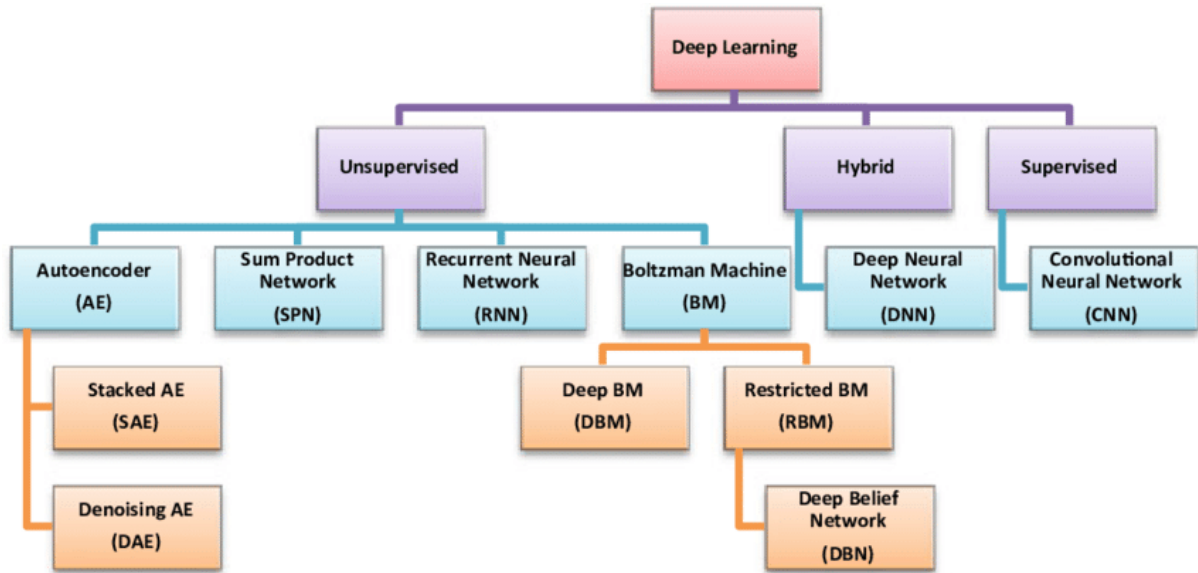


Figure 2 Classification of deep learning techniques

Supervised learning methods have the advantage of providing highly accurate and reliable predictions because the models learn from labelled data. This allows for precise identification of customer segments based on pre-defined criteria, leading to more targeted marketing strategies (Li et al., 2023). However, the major limitation of supervised methods is their dependency on labelled data, which can be expensive to obtain and sometime may not reflect all segment in the data. Additionally, supervised learning models may not generalize well to new or unseen data if the training set is not representative of the overall customer base.

Unsupervised learning methods, on the other hand, do not require labelled data, making them more flexible and scalable for customer segmentation. These methods can uncover hidden patterns and relationships in the data that might not be apparent through supervised learning (Sarker, 2021). This capability is particularly valuable in customer segmentation, where the goal is to discover natural groupings of customers based on their behaviours and preferences. The primary advantage of unsupervised methods is their ability to handle large and complex datasets without the need for extensive labelling. However, the main limitation is that the results can be less interpretable, and the quality of the segmentation heavily depends on the chosen algorithm and the nature of the data.

In the context of customer segmentation, unsupervised methods are often preferred because they can reveal new and unexpected segments that might not be defined in a labelled dataset. For instance, unsupervised learning can identify emerging customer trends or niche markets that have not been previously considered. This exploratory nature of unsupervised learning makes it a powerful tool for businesses looking to innovate and stay ahead of market trends (Li et al., 2023). Several deep unsupervised methods have gained prominence in customer segmentation research. Autoencoders, a type of neural network designed to learn efficient data encodings, have been successfully applied to reduce the dimensionality of customer data and uncover latent features. A study by Huang, (2023) demonstrated that autoencoder-based segmentation improved customer retention rates by 10% compared to traditional clustering methods. Generative Adversarial Networks (GANs) have also shown promise in customer segmentation. These models can generate synthetic customer profiles, helping to augment limited datasets and identify subtle patterns that distinguish different customer groups.

2.1 Autoencoders

Autoencoders are unsupervised deep learning models that can effectively compress and reconstruct customer data for segmentation purposes (Charte et al., 2020). Karim et al., (2020) emphasized the fundamental principle behind autoencoders is to compress the input data into a lower-dimensional representation and then reconstruct it as closely as possible to the original input. This process forces the network to learn the most salient features of the data, effectively performing dimensionality reduction while preserving critical information.

The architecture of an autoencoder typically consists of three main components as shown in figure 3: an encoder, a latent space, and a decoder as described by Wu and Wang, (2022). The encoder compresses the input data into a lower-dimensional representation, often referred to as the latent space or bottleneck layer. This latent space forces the network to learn a compact representation of the input data, capturing its most essential features (Bishayee, 2023). The decoder then attempts to reconstruct the original input from this compressed representation. Through the training the network to minimize the difference between the input and its reconstruction, the autoencoder learns to capture the underlying structure of the data (Ekaba, 2019).

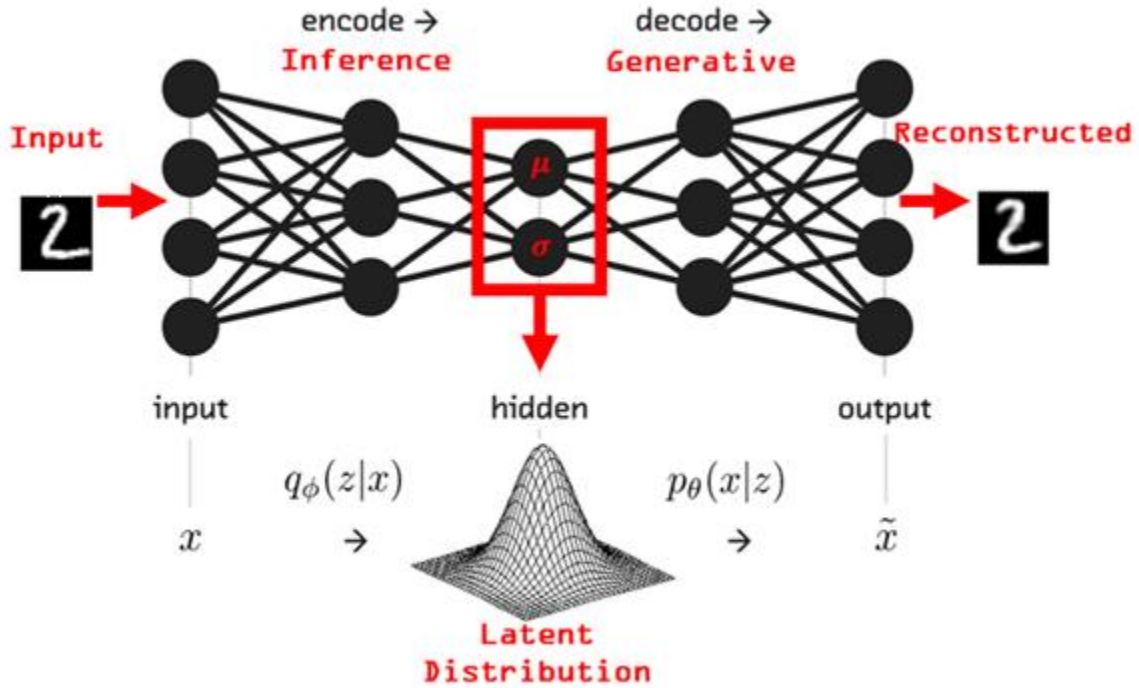


Figure 3 Architecture of autoencoder model

In customer segmentation, autoencoders offer several advantages. First, they can handle mixed attribute types commonly found in customer data, including numerical, categorical, and even textual information (Choi et al., 2024). This versatility allows for a more comprehensive analysis of customer characteristics. Second, autoencoders excel at capturing nonlinear relationships within the data, which is particularly valuable in customer behaviour analysis where interactions between various factors often exist (Kufel et al., 2023).

The scalability of autoencoders to large datasets is another significant advantage in customer segmentation as stated by Alkhayrat et al., (2020) who noted that as businesses accumulate large amounts of customer data from various sources, including transaction histories, web browsing behaviour, and social media interactions, autoencoders can efficiently process and extract meaningful insights from these high-dimensional datasets. This scalability ensures that the segmentation model remains effective as the volume and complexity of customer data grow (Yoseph et al., 2020).

Moreover, autoencoders can be extended to more variants that offer additional benefits for customer segmentation. VAEs, for instance, introduce a probabilistic approach to the encoding process as noted by Zhai et al., (2018) that allows for the generation of new data points and provides a measure of uncertainty in the segmentation. This probabilistic framework can be particularly useful in understanding the fuzziness of segment boundaries and in generating synthetic customer profiles for strategy testing (Xu et al., 2021). Denoising Autoencoders, another variant, are trained to reconstruct clean input from corrupted versions, making them robust to noise in customer data (Davila et al., 2021). This property is valuable in real-world scenarios where customer data often contains inconsistencies or missing values (Kaufmann et al., 2020). Sparse Autoencoders, which impose sparsity constraints on the hidden layers, can lead to more interpretable features, potentially aligning with specific customer attributes or behaviours (Lei et al., 2024). The application of autoencoders in customer segmentation extends beyond mere clustering. Huang and Liao, (2020) noted that they can be used for anomaly detection, identifying unusual customer behaviours that might indicate fraud or special opportunities. In recommendation systems, autoencoders can learn latent representations of customer preferences, enabling more personalised product suggestions. Furthermore, the latent space learned by autoencoders as stated by Buckner-Petty et al., (2018) can serve as a powerful feature extraction step for subsequent supervised learning tasks, such as predicting customer churn or lifetime value. Despite these advantages, there are few existing research on autoencoder-based unsupervised machine learning in customer segmentation. One significant limitation is the small number of studies that have applied these techniques in real-world business contexts as emphasized earlier by Shams et al., (2023).

Deep learning-based feature extraction techniques, such as AE and VEA, have demonstrated superior capability in capturing complex nonlinear relationships within raw data compared to traditional data transformation methods. These traditional methods include PCA and its variants, as discussed by Kherif and Latypova (2020), Isometric Mapping (ISOMAP) as explored by Yousefi et al. (2024), and Multi-Dimensional Scaling (MDS) examined by Kalia and Paul (2021). In the integration of deep learning with classical clustering, two main approaches have emerged in prior research. The sequential approach, which is more prevalent,

involves using a deep Autoencoder to learn low-dimensional latent-space representations before applying clustering methods. This approach has been highlighted in the work of Nicholas et al. (2021). Several types of Deep Autoencoders have been applied to customer segmentation tasks, including standard AE as used by Nguyen (2021), Stacked Autoencoders (SAE) as implemented by Alkhayrat, Aljnidi, and Aljoumaa (2020), and VAE as explored by Mancisidor et al. (2021). The alternative approach, which is less common in customer segmentation, is the synchronous method. This approach, as described by Yang et al. (2017), involves the simultaneous use of deep neural networks for learning both feature representation and cluster assignment.

2.2 Hyperparameters in autoencoder

Autoencoders are characterized by various hyperparameters that significantly influence their performance, and their values must be carefully defined prior to training. These hyperparameters include the number of hidden layers, number of neurons in each layer, size of the latent space, activation function, objective function, and optimization algorithm (Alarsan and Younes, 2021). As describe by Li et al., (2021), number of hidden layers determines the network's depth and capacity to capture distinct data patterns, but adding more layers can introduce optimization challenges and elevate the risk of overfitting. The number of neurons in each layer governs the network's data representation capacity and is typically set before training, with a higher count amplifying the network's capacity but also increasing the risk of overfitting and complicating optimization (Li and Li, 2021). The size of the latent space, represented by the bottleneck layer, is a critical hyperparameter that balances model complexity and performance. The activation function used in the bottleneck layer plays a crucial role in the autoencoder's performance, with common functions including sigmoid, tanh, ReLU, and SELU (Tomar, 2022). Sigmoid is suitable for binary or probability-like outputs, but can suffer from vanishing gradients, while ReLU is more suitable for deep autoencoders due to its non-saturating nature. Tanh is beneficial for feature scaling and zero-centred outputs but can also suffer from vanishing gradients. SELU offers better performance and self-normalizing properties, making it advantageous for bottleneck layers. Additionally, the objective function, also known as the loss function, is critical for training the autoencoder by minimizing the

distinction between input and output data (Lopez Pinaya et al., 2020). Common objective functions include Mean Squared Error (MSE) and Binary Cross-Entropy (BCE), with MSE being the predominant function and BCE employed for binary input data. The optimization algorithm is another critical hyperparameter, utilized to minimize the objective function during training, with popular algorithms including Stochastic Gradient Descent (SGD), Adam, and Adagrad (Tian, Zhang, and Zhang, 2023). The choice of optimization algorithm depends on dataset size, model complexity, loss function type, and computational resources. Furthermore, the learning rate, a hyperparameter, dictates the step size during optimization and influences weight and bias updates and convergence speed (Azam et al., 2024). The number of epochs affects model accuracy and the risk of overfitting, while batch size affects gradient noise and optimization efficiency. These interconnected hyperparameters necessitate careful selection for optimal performance, often requiring experimentation and hyperparameter tuning during training to build an effective autoencoder model.

2.4. Summary of factors for effective Autoencoder model

Recent literature identifies several key factors for effectively building autoencoder models, including architecture, regularization techniques, training strategies, data preprocessing and augmentation, and hyperparameter tuning settings.

Firstly, the architecture of the autoencoder, encompassing depth, width, layer types, and bottleneck size, is crucial for performance. Additionally, regularization techniques such as dropout, weight regularization, and denoising enhance the model's generalizability.

Next, effective training strategies involve selecting appropriate loss functions, utilizing advanced optimization algorithms, and applying batch normalization to stabilize and accelerate training. Furthermore, the quality of data preprocessing, including normalization, standardization, and augmentation, significantly impacts model performance by ensuring consistent data scales and enhancing robustness.

Hyperparameter tuning is also essential, focusing on aspects such as learning rate, batch size, and the number of training epochs to find the optimal configuration for the autoencoder. Lastly,

evaluation metrics like reconstruction error, latent space quality, and application-specific metrics ensure a thorough performance assessment.

2.5 Research Gap

Despite the increasing popularity of Autoencoders in customer segmentation, a significant knowledge gap persists in understanding the relative merits of traditional Autoencoders and VAE in this context. The existing literature is fragmented, with most studies focusing on either Autoencoders or VAE, but not on a comprehensive comparison of their performance in customer segmentation tasks. Furthermore, the role of PCA as a preprocessing step before clustering has not been thoroughly explored. While PCA is commonly used to reduce dimensionality, its impact on the clustering results and the subsequent customer segmentation has not been systematically investigated. To address these limitations, this study aims to fill the research gap by providing a comprehensive comparison of traditional Autoencoders and VAE in customer segmentation, including the impact of PCA as a preprocessing step, hyperparameter tuning, and a detailed evaluation of the models using a range of relevant metrics.

3 Practical Research Methodology

3.1 Introduction

This chapter focuses on developing and evaluating the machine learning model for segmenting customers based on the autoencoder method. This entails outlining the procedures for data collection activities, data pre-processing, data modelling and data evaluation. Among the available frameworks, the CRISP-DM (Cross-Industry Standard Process for Data Mining) was chosen due to its holistic, adaptable and cyclical scheme for data mining with a clear guideline for planning and executing a data mining project and it makes sure that none of the stages from understanding data to deployment is missed (Schröder et al., 2021). This project implemented Python as the main language and incorporated TensorFlow & Keras for modelling, Pandas for data preprocessing, and Matplotlib for visualization. Google Colab with Tesla T4 GPU has been used for computational resources because it is easily available and powerful processing capabilities, and the code of this implementation is shown in Appendix A.

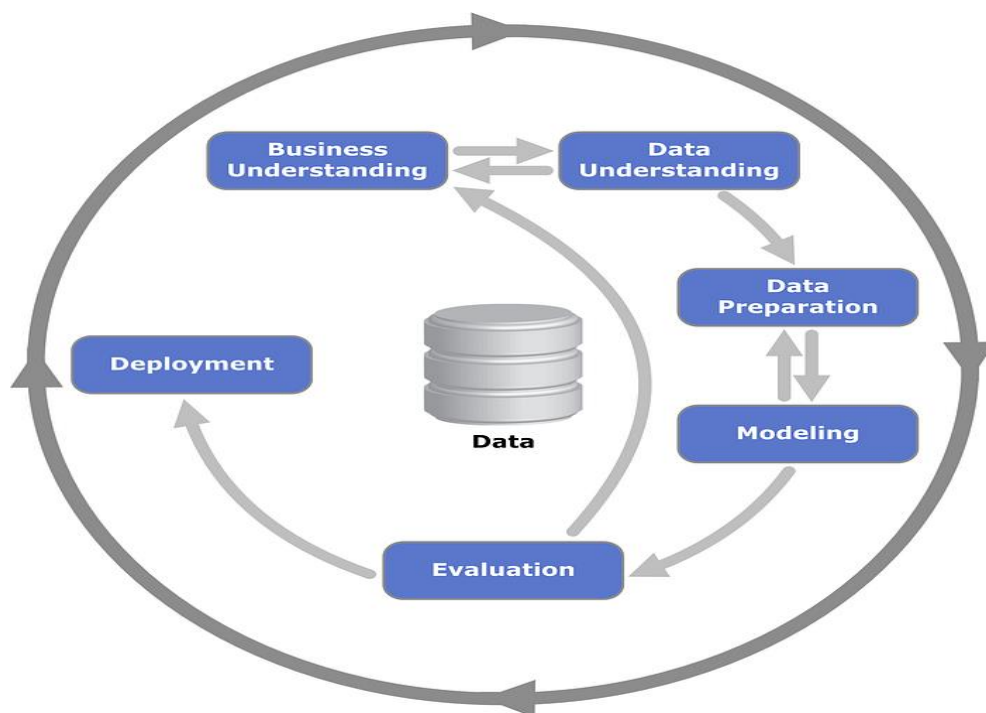


Figure 4 CRISP-DM for Data Science (Schröder et al., 2021)

3.2 Data Understanding

This study makes use of consumer datasets that are freely accessible on Kaggle. Kaggle offers a diverse range of datasets of different domain and related to consumer behaviour and segmentation making it an ideal source for this research. The dataset is a collection of consumer data, including demographics, past purchases, online activities, and other essential customer characteristics ([Customer Personality Analysis \(kaggle.com\)](https://www.kaggle.com/datasets/charanasomboon/customer-personality-analysis)). Exploratory analysis is carried out to comprehend the properties and characteristics of the data after obtaining the datasets from Kaggle. As described Charanasomboon and Viyanon, (2019), analysing data dimensions, variable types, and statistical summaries are necessary to obtain a better understanding of the distribution and variability of customers features. In data modelling, data exploration is the initial stage where features and preliminary patterns of a dataset are analysed using statistical methods and visual aids (Kaliyadan and Kulkarni, 2019). The dataset consists of 2240 records and 29 columns of client data, each representing a distinct customer profile as in Table 1 below.

Table 1 Data summary and description

Column	Description	Dtype
ID	Customer's unique identifier	int64
Year Birth	Customer's birth year	int64
Education	Customer's education level	object
Marital Status	Customer's marital status	object
Income	Customer's yearly household income	float64
Kidhome	Number of children in customer's household	int64
Teenhome	Number of teenagers in customer's household	int64
Dt Customer	Date of customer's enrollment with the company	object
Recency	Number of days since customer's last purchase	int64
Complain	1 if the customer complained in the last 2 years, 0 otherwise	int64
MntWines	Amount spent on wine in last 2 years	int64

MntFruits	Amount spent on fruits in last 2 years	int64
MntMeatProducts	Amount spent on meat in last 2 years	int64
MntFishProducts	Amount spent on fish in last 2 years	int64
MntSweetProducts	Amount spent on sweets in last 2 years	int64
MntGoldProds	Amount spent on gold in last 2 years	int64
NumDealsPurchases	Number of purchases made with a discount	int64
AcceptedCmp1	1 if customer accepted the offer in the 1st campaign, 0 otherwise	int64
AcceptedCmp2	1 if customer accepted the offer in the 2nd campaign, 0 otherwise	int64
AcceptedCmp3	1 if customer accepted the offer in the 3rd campaign, 0 otherwise	int64
AcceptedCmp4	1 if customer accepted the offer in the 4th campaign, 0 otherwise	int64
AcceptedCmp5	1 if customer accepted the offer in the 5th campaign, 0 otherwise	int64
Response	1 if customer accepted the offer in the last campaign, 0 otherwise	int64
NumWebPurchases	Number of purchases made through the company's website	int64
NumCatalogPurchases	Number of purchases made using a catalogue	int64
NumStorePurchases	Number of purchases made directly in stores	int64
NumWebVisitsMonth	Number of visits to company's website in the last month	int64
Z_CostContact	Dummy field	int64
Z_Revenue	Dummy field	int64

In addition, a bivariate analysis is carried out to ascertain the correlation between variables to prevent multicollinearity. Apart from examining trends in customer behaviour, the distribution of different customer attributes is examined, with an emphasis on evaluating their frequency,

relevance, and possible influence on segmentation. Features including purchase frequency, average order value, product preferences, and participation with marketing channels are given special consideration as they may be indicative of distinct consumer categories. This analysis helps inform the understanding of potential customer segments that may emerge from the unsupervised learning process.

3.3 Data preparation

Data preparation also known as preprocessing is an essential part of machine learning as it involves removing the missing values, outliers, and inconsistencies in the dataset as well as transforming the data to create a dataset that is ready for modelling. These operations include data acquisition, data cleaning, data augmentation, and data formatting, which prepares data for training and model testing (Stedman, 2022). Skillfloor, (2023), stated that accurate and quality data preparation can lead to high accuracy, reliability, and efficiency of the predictions, as well as a decrease in risk and cost of data quality problems and model mistakes.

3.3.1 Data Cleaning

Data cleaning is the process of identifying the errors in the customer dataset before initiating the analysis. This involves deletion, revision, or modification of data, which is wrong, partial, replicated or otherwise misleading. Some of the factors that affect the analysis of data include the presence of missing values which happen when there is no value recorded in the variable for a specific observation. During data collection and understanding it discovered that there are missing values in the income columns.

The first step in data cleaning involved examining the dataset for missing values and removing any incomplete records to ensure the completeness of the data. Outlier removal was also performed by establishing an income threshold, specifically excluding any record with an income greater than 600,000 to minimize the impact of extreme values on the analysis. Additionally, the ID, Year_Birth, Dt_Customer, Z_CostContact, and Z_Revenue was removed from the dataset as they were deemed non-essential for clustering analysis. In the data transformation step, categorical data segmentation was performed to simplify the dataset. For education levels, the data was segmented into three broader categories: 'PostGraduation'

(combining 'PhD' and 'Master'), 'Graduation', and 'UnderGraduation' (combining '2n Cycle' and 'Basic'). This simplification reduced the number of unique categories and made the data more manageable. Similarly, marital status categories were consolidated into two groups: 'Together' (including 'Married' and 'Together') and 'Alone' (including 'Single', 'Divorced', 'Widow', 'Absurd', and 'YOLO') to streamline the dataset further.



```
# @title Data processing and feature Engineering
#Drop NA values
clustering_df.dropna(inplace=True)
#Dropping the outlier by setting a cap on income
clustering_df = clustering_df[clustering_df.Income<600000]

#New feature for total spendings on various items and Time with the company
def get_age(birthyear):
    return date.today().year - birthyear
clustering_df["Age"] = clustering_df.Year_Birth.apply(get_age)

clustering_df["Spent"] = clustering_df["MntWines"] + clustering_df["MntFruits"] + \
    clustering_df["MntMeatProducts"] + clustering_df["MntFishProducts"] + \
    clustering_df["MntSweetProducts"] + clustering_df["MntGoldProds"]

clustering_df['Time_With_Company'] = pd.to_datetime(clustering_df.Dt_Customer, dayfirst = True, format = '%d-%m-%Y')
clustering_df['Time_With_Company'] = pd.to_numeric(clustering_df.Time_With_Company.dt.date\
    .apply(lambda z: (date.today() - z)).dt.days, downcast = 'integer') / 30

#New feature for total members in the household
clustering_df['Family_Size'] = clustering_df.Kidhome + clustering_df.Teenhome + clustering_df.Marital_Status.replace({'Alone':1,'Together':2})

#Feature pertaining parenthood
clustering_df['Is_Parent'] = np.where((clustering_df.Kidhome + clustering_df.Teenhome)>0,1,0)

#Dropping some of the features
clustering_df.drop(['ID','Year_Birth','Dt_Customer','Z_CostContact','Z_Revenue'],axis=1,inplace=True)
```

Figure 5 section of preprocessing

3.3.2 Feature selection and engineering

Enhancing the predictive capacity of machine learning models is mostly dependent on feature engineering and selection. Finding the most pertinent customer characteristics for segmentation is a crucial step in customer personality analysis since it lowers dimensionality and boosts model performance. Several methods are taken into consideration for consumer segmentation with autoencoders. To avoid multicollinearity problems, duplicated features are eliminated, and highly correlated features are found using correlation analysis.

Feature engineering was an essential part of the data transformation process. A new feature, Spent, was created to represent the total amount spent by each customer across various product categories such as wines, fruits, meat products, fish products, sweets, and gold products. This feature provides insights into the overall spending behaviour of customers. Another new feature, Family Size, was introduced to represent the total number of members in a customer's household by adding the number of kids (Kidhome), teenagers (Teenhome), and a value indicating marital status (1 for 'Alone' and 2 for 'Together'). Additionally, the Is_Parent feature was created to indicate whether a customer has children or teenagers at home (1 if they have kids or teenagers, 0 otherwise), helping to segment customers based on parenthood status.

Categorical variables such as Education and Marital Status were then converted into numerical form using label encoding, which is essential for machine learning algorithms that require numerical input. Finally, feature scaling was performed using the StandardScaler to standardize the features, ensuring they have a mean of zero and a standard deviation of one. Standardizing the data ensures that all features contribute equally to the analysis and prevents features with larger scales from dominating the results. Through these preprocessing steps, the data was transformed into a format suitable for clustering analysis, ensuring that the dataset is clean, consistent, and ready for the application of machine learning techniques.

Furthermore, features are ranked according to how important they are in differentiating between client categories using feature importance approaches like PCA. For data reduction, PCA was employed to transform the standardized data into a more manageable format. PCA was used to reduce the dimensionality of the dataset while retaining the most significant variance. Specifically, the data was transformed into three principal components, resulting in a new DataFrame with columns labelled pca1, pca2, and pca3. This transformation helps to capture the essential features of the data in fewer dimensions, facilitating more efficient analysis and clustering.

3.4 Modelling

For this unsupervised task for customer segmentation, the modelling entails developing algorithms that can recognise patterns and classify customer without the need for pre-labelled

data. The three main objectives of this procedure are cluster analysis, dimensionality reduction, and feature extraction which are then clustered. To achieve this, it involves evaluating different unsupervised learning techniques and choosing the one that best addresses the customer segmentation task. This research focuses on the autoencoder methods (its base model and variational autoencoder) due to its ability to handle high-dimensional data and learn meaningful representations, considering several experiments methods to see which performs better for customer segmentation.

3.4.1 Autoencoder Base model Architecture

An autoencoder is a type of artificial neural network that can be used to automatically learn effective representations of data. The autoencoder is a great option for unsupervised consumer segmentation due to its capacity to identify patterns in customer data. Autoencoders can determine the most significant characteristics and connections in a dataset by learning to recreate the input data. The autoencoder model consists of an input layer, an encoding layer, a bottleneck layer, a decoding layer, and an output layer. As shown in figure 6, the input layer accepts data with a dimensionality equivalent to the input dimension, which, for this study, is set to 784. The encoding layer, a dense layer, reduces the input dimensionality to the encoding dimension, here set to 29, 500 and 2000, and applies the ReLU activation function. The bottleneck layer further reduces the encoding dimension by half, employing another dense layer with ReLU activation. The decoding layer, another dense layer with ReLU activation, expands the bottleneck dimension back to the encoding dimension. Finally, the output layer, a dense layer with sigmoid activation, restores the data to its original input dimensionality. The autoencoder model is compiled using the Adam optimizer and mean squared error (MSE) as the loss function. These model architectures are shown in figure 6 and code shown in appendix A.

Model: "model"

Layer (type)	Output Shape	Param #
input_1 (InputLayer)	[(None, 29)]	0
dense (Dense)	(None, 500)	15000
dense_1 (Dense)	(None, 500)	250500
dense_2 (Dense)	(None, 2000)	1002000
dense_3 (Dense)	(None, 2000)	4002000
dense_4 (Dense)	(None, 500)	1000500
dense_5 (Dense)	(None, 500)	250500
dense_6 (Dense)	(None, 29)	14529

=====
Total params: 6535029 (24.93 MB)
Trainable params: 6535029 (24.93 MB)
Non-trainable params: 0 (0.00 Byte)
=====

Figure 6 Model Architecture for Autoencoder

3.4.2 Variational Autoencoder model Architecture

The variational autoencoder (VAE) model architecture begins with an input layer designed to accept data with the same dimensionality as the input dimension. A dense layer with ReLU activation is used as the hidden layer, which reduces the dimensionality to the encoding dimension, here set to 64. From this hidden layer, two separate dense layers are created to generate the mean (z_mean) and the logarithm of the variance (z_log_var) for the latent variables, each having a dimensionality of encoding dimension divided by two. The model incorporates a custom sampling function, which uses these latent variables to sample from a normal distribution to create a latent vector z . The decoder part of the VAE starts with a dense layer with ReLU activation, expanding the latent vector z back to the encoding dimension. This is followed by another dense layer with sigmoid activation, which decodes the data back to the

original input dimensionality. The overall VAE model is compiled with a custom loss function that combines the reconstruction loss (mean squared error) and the Kullback-Leibler divergence, which measures the difference between the learned latent variable distribution and the standard normal distribution. The model is then optimized using the Adam optimizer as shown in figure 7.

Model: "model_s"

Layer (type)	Output Shape	Param #	Connected to
input_5 (InputLayer)	(None, 29)	0	[]
dense_22 (Dense)	(None, 500)	15000	['input_5[0][0]']
dense_23 (Dense)	(None, 250)	125250	['dense_22[0][0]']
dense_24 (Dense)	(None, 250)	125250	['dense_22[0][0]']
lambda_3 (Lambda)	(None, 250)	0	['dense_23[0][0]', 'dense_24[0][0]']
dense_25 (Dense)	(None, 500)	125500	['lambda_3[0][0]']
dense_26 (Dense)	(None, 29)	14529	['dense_25[0][0]']
tf.__operators__.add_6 (TF OpLambda)	(None, 250)	0	['dense_24[0][0]']
tf.math.square_3 (TFOpLambda)	(None, 250)	0	['dense_23[0][0]']
tf.math.subtract_6 (TFOpLambda)	(None, 250)	0	['tf.__operators__.add_6[0][0]', 'tf.math.square_3[0][0]']
tf.math.exp_3 (TFOpLambda)	(None, 250)	0	['dense_24[0][0]']
tf.convert_to_tensor_3 (TF OpLambda)	(None, 29)	0	['dense_26[0][0]']
tf.cast_3 (TFOpLambda)	(None, 29)	0	['input_5[0][0]']
tf.math.subtract_7 (TFOpLambda)	(None, 250)	0	['tf.math.subtract_6[0][0]', 'tf.math.exp_3[0][0]']
tf.math.squared_difference_3 (TFOpLambda)	(None, 29)	0	['tf.convert_to_tensor_3[0][0]', 'tf.cast_3[0][0]']
tf.math.reduce_sum_3 (TFOpLambda)	(None,)	0	['tf.math.subtract_7[0][0]']
tf.math.reduce_mean_6 (TFOpLambda)	(None,)	0	['tf.math.squared_difference_3[0][0]']
tf.math.multiply_3 (TFOpLambda)	(None,)	0	['tf.math.reduce_sum_3[0][0]']
tf.__operators__.add_7 (TF OpLambda)	(None,)	0	['tf.math.reduce_mean_6[0][0]', 'tf.math.multiply_3[0][0]']
add_loss_3 (AddLoss)	(None,)	0	['tf.__operators__.add_7[0][0]']

Figure 7 Model Architecture for VAE

3.4.3 Model Training

The training phase involves fitting the model on the training data, using the input data as both features and targets. This self-supervised approach allows the autoencoders to learn the

underlying structure of the data without the need for labelled examples. During training, the model iteratively adjusts its weights to minimize the reconstruction error, effectively learning to compress and decompress the input data while preserving its essential characteristics. The validation phase monitors reconstruction errors on a validation set to prevent overfitting. This step is crucial to ensure that the model generalizes well to unseen data and does not simply memorize the training examples. Through the monitoring the validation error, it is possible to implement early stopping or other regularization techniques to improve the model's performance and generalizability.

3.4.4 Hyperparameter Tuning of the Autoencoders

To optimize the autoencoder's performance, Grid Search is used to tune various hyperparameters as it includes the number of layers in the encoder and decoder, which determines the depth of the network and its capacity to learn complex representations. The number of neurons in each layer affects the model's ability to capture and preserve information at different levels of abstraction. Activation functions, such as ReLU, tanh, and sigmoid, introduce non-linearity into the network, allowing it to learn more complex patterns. ReLU formula is

$$f(x) = \max(0, x)$$

Both the ReLU function and its derivative are monotonic. If the function receives any negative input, it returns 0; however, if the function receives any positive value x , it returns that value. As a result, the output has a range of 0 to infinite.

Other important hyperparameters include the learning rate, which controls the step size during optimization, the batch size, which affects the stochasticity of the training process and the memory requirements, and the number of epochs, which determines how many times the model iterates over the entire dataset during training were considered. The Grid Search process is used systematically evaluates different combinations of these hyperparameters to find the configuration that yields the best performance on a validation set.

3.5 Summary of Experiment conducted

In this study series of experiment was conducted to investigate the effectiveness of different machine learning models for customer segmentation as should in figure 8. The primary models evaluated were a base autoencoder and a variational autoencoder (VAE). Additionally, traditional clustering methods, including K-means, DBSCAN, and hierarchical clustering, were also applied for comparison. The experiments were designed to assess the models' ability to segment customers based on demographic and behavioural data. The first experiment that was conducted is without dimensionality using PCA for the selected model as well as with PCA. This was done with and without hyperparameter tuning to see how each of the factor affect the clustering methods performed based on the performance metric. In addition to the autoencoder and VAE models, traditional clustering methods were evaluated for comparisons this include, K-means, DBSCAN, and hierarchical clustering.

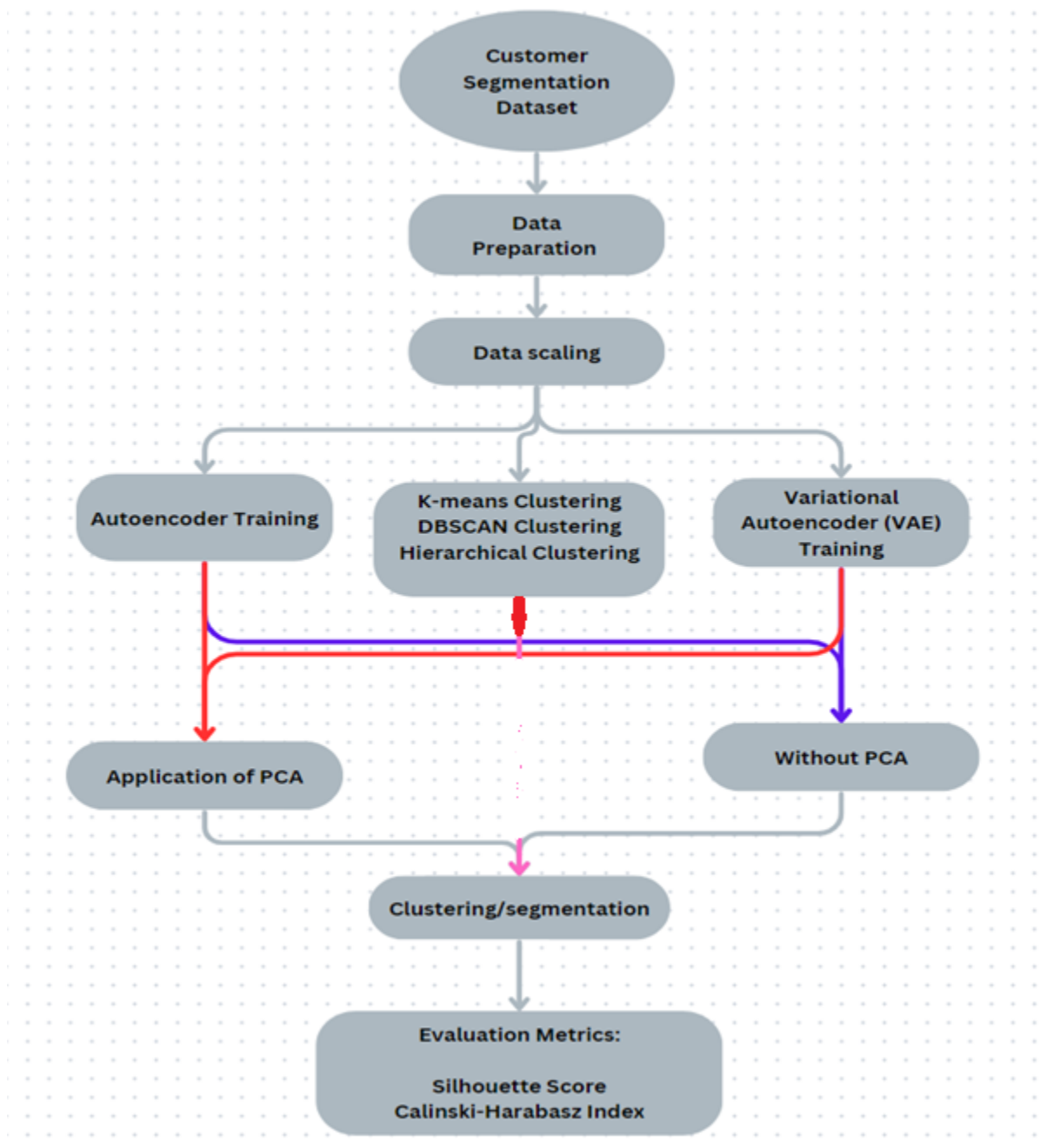


Figure 8 Summary of implementation

3.6 Segmentation

To explore the impact of dimensionality reduction, PCA is applied to the original scaled data, the encoded data from the autoencoder, and the encoded data from the VAE, reducing them to three principal components. Clustering is then performed on the encoded data from both the

autoencoder and VAE models, as well as their PCA-transformed versions. Traditional clustering methods like K-means and DBSCAN are also applied to the original and transformed data to provide a basis for comparison. This comprehensive approach allows us to determine which method performs better based on the selected evaluation metric. The evaluation of clustering performance involves comparing the results from the autoencoder, VAE, and traditional clustering methods with and without PCA. The results are assessed using the chosen metric to identify the most effective clustering approach for the given dataset. This comparison helps highlight the strengths and weaknesses of each method and the impact of dimensionality reduction on clustering performance. Following the clustering experiments, hyperparameter tuning is conducted for both the autoencoder and VAE models to optimize their performance. This involves systematically adjusting various hyperparameters and evaluating the models on the training and test data to identify the best parameter configurations. The tuning process ensures that each model achieves optimal clustering performance, providing a robust comparison between methods.

3.7 Evaluation

Model evaluations are a way to measure the quality of a model, and its performance and are used to determine the effectiveness of a model and its suitability for specific tasks. The quality, dependability, and usefulness of the produced customer segments are evaluated by the model evaluation, which is based on customer personality analysis. To make sure the segmentation findings are both statistically sound and useful for commercial objectives, this step entails an analysis of the model's performance using a variety of metrics and validation methodologies. The assessment procedure directs choices about model implementation and refinement, acting as a vital link between the technical aspects of the analysis and its real-world application in marketing strategy. For clustering-based customer segmentation, traditional classification metrics such as accuracy, precision, recall, and F1-score are not directly applicable due to the unsupervised nature of the task. Silhouette score, Elbow method, Davies-Bouldin index, and Calinski-Harabasz index are other wise used. These metrics ensure that the clustering results are both cohesive and distinct, which is crucial for meaningful segmentation. The Davies-Bouldin index (DBI) measures the average similarity ratio of each cluster with its most similar

cluster, where a lower DBI indicates better clustering. This index considers both the scatter within clusters and the separation between clusters. As Singh et al., (2020) describe, the DBI is calculated based on the ratio of the sum of within-cluster scatter to between-cluster separation for each cluster. The DBI is advantageous because it provides a relative measure of cluster separation and compactness, thus offering a comprehensive view of clustering performance. However, a potential drawback is its sensitivity to the definition of cluster centroids and the distance metric used, as noted by Xiao et al., (2017).

Silhouette Score

The silhouette score is a widely used metric for evaluating the consistency and separation of clusters. It measures how similar an object is to its own cluster compared to other clusters, providing a clear indication of the clustering quality. Specifically, a high silhouette score indicates that data points are well-matched to their own cluster and poorly matched to neighbouring clusters (Shahapure and Nicholas, 2020). The silhouette score is particularly valued for its ability to provide an intuitive graphical representation of clustering performance, making it a popular choice among researchers for initial cluster validation. The Elbow method is a heuristic used in determining the optimal number of clusters in a dataset. It involves plotting the explained variation as a function of the number of clusters and looking for the "elbow point," where the rate of increase in explained variance slows down. This point suggests the optimal number of clusters. Sarvani et al., (2023) highlighted that the Elbow method is useful for identifying the number of clusters that provide a meaningful balance between complexity and explanation power. Despite its simplicity, the method can be subjective as the exact location of the elbow can sometimes be uncertain, which is a limitation noted by Nicolis et al., (2024). Nevertheless, it remains a fundamental tool for determining the number of clusters to use in further analysis.

The silhouette score for a data point (i) is calculated as:

$$S(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}$$

Where $a(i)$ is the average distance between i and all other points in the same cluster, and $b(i)$ is the minimum average distance between i and points in another cluster, of which i is not a member. The silhouette score ranges from -1 to 1, with values close to 1 indicating well-clustered data and values close to -1 indicating poor clustering.

Calinski-Harabasz Index

The Calinski-Harabasz index, also known as the Variance Ratio Criterion, evaluates the ratio of the sum of between-cluster dispersion to within-cluster dispersion. Higher values of the index indicate better-defined clusters. Das, (2022) introduced this index, demonstrating that it effectively balances cluster cohesion and separation. The Calinski-Harabasz index is particularly favoured for its computational efficiency and robustness across different clustering scenarios. Despite its effectiveness, some researchers, such as Ekemeyong et al., (2023) have pointed out that the index might favour solutions with a higher number of clusters, necessitating careful interpretation and complementary evaluation methods. The Calinski-Harabasz index is calculated as:

$$CH = \frac{BCSS/(k - 1)}{WCSS/(n - k)}$$

Where:

- BCSS is the Between-Cluster Sum of Squares
- WCSS is the Within-Cluster Sum of Squares
- k is the number of clusters
- n is the total number of data points

Justification for Metric selection

The silhouette score and Calinski-Harabasz Index are essential metrics that will be used in the study for evaluating customer segmentation models created using the Autoencoder method. The reason behind this is that for Silhouette Score, it offers an intuitive graphical representation of clustering quality, making it easier for stakeholders to understand and interpret the results.

Secondly, it is particularly suitable for high-dimensional data, which is often the case in customer segmentation tasks involving numerous features and purchase behaviours. Lastly, the silhouette score can be calculated regardless of the clustering algorithm used, making it a versatile tool for comparing different clustering methods, including those integrated with Autoencoders. This versatility is important in ensuring that the best possible clustering method is selected for identifying customer segments (Bhardwaj, 2020). On the other hand, Calinski-Harabasz Index is computationally efficient and can handle large datasets, which is often necessary in customer analytics. Additionally, it provides a clear numerical value that can be used to compare different clustering models, ensuring that the most effective model is chosen. This is particularly important when employing models like Autoencoders, where understanding the clustering performance can be more challenging. Furthermore, the index's focus on variance ensures that clusters are not only distinct but also internally consistent, which is essential for deriving meaningful customer segments that can inform business strategies (Lima and Cruz, 2020).

4 Analysis of Results

4.1 Experiment one Model Results (without PCA)

In this experiment, two models were trained and evaluated: a base autoencoder and a variational autoencoder (VAE) as shown in figure 9 and 10. The autoencoder was designed to learn a compressed representation of the customer data, using several hidden layers for encoding and decoding. The VAE, a generative model, was also trained to learn the latent structure of the data while incorporating stochastic elements. Both models aimed to segment customers based on their demographic and behavioural data.

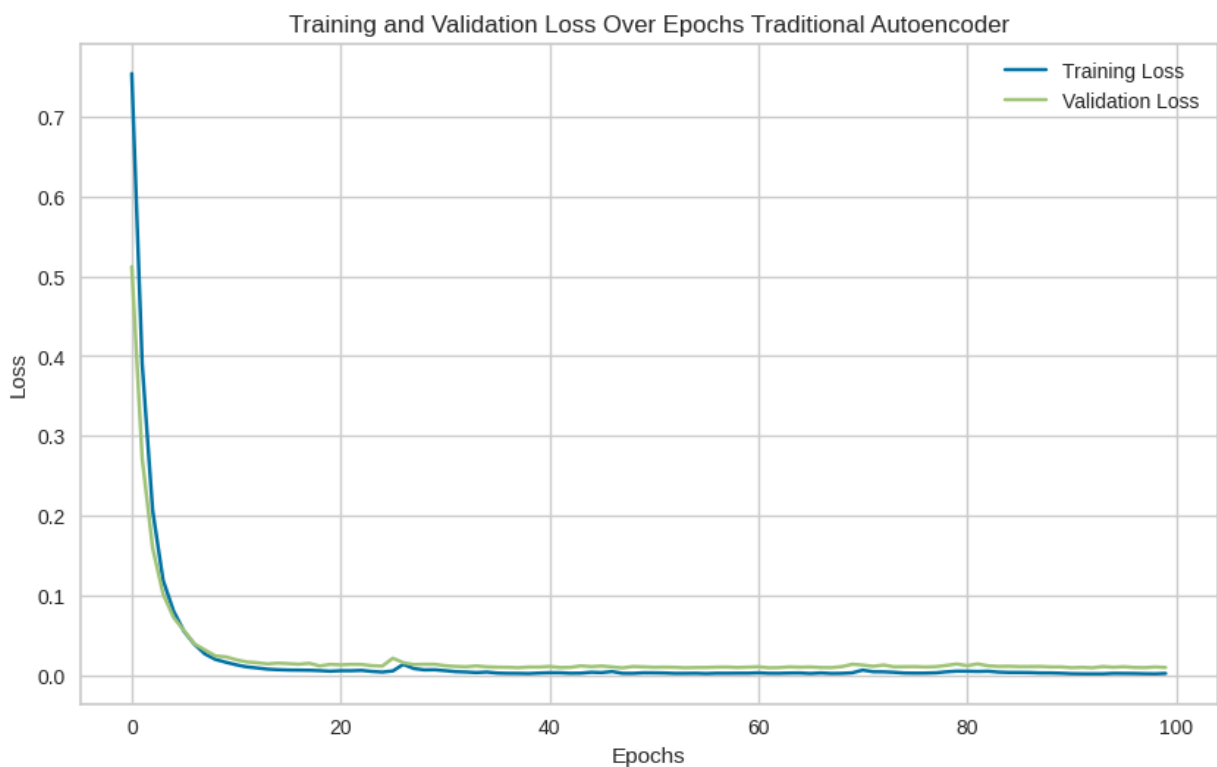


Figure 9 Training and validation loss over Epochs for the base autoencoder model

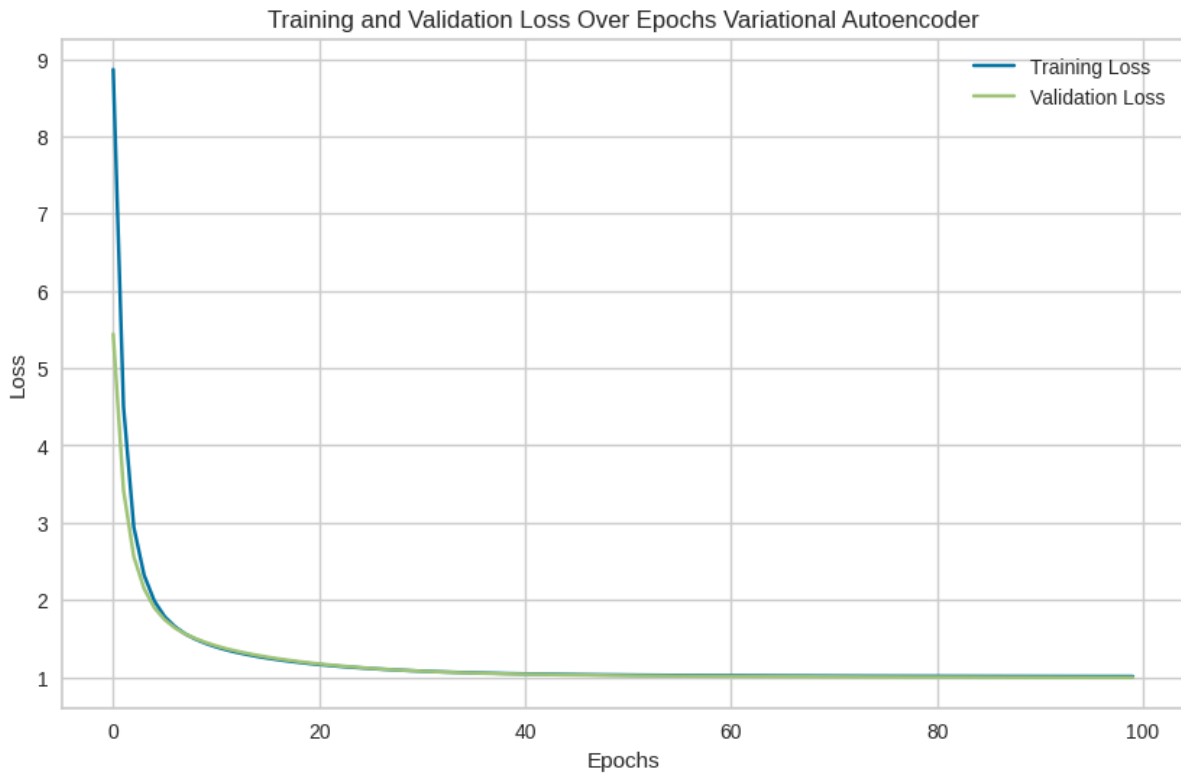


Figure 10 Training and validation loss over Epochs for the base VAE model

Figure 9 and 10 shows the training and validation loss over epochs for the autoencoder and VAE respectively. The model demonstrates rapid initial learning, with both training and validation losses decreasing sharply in the first few epochs. The loss starts high (around 0.7 and 9) and quickly drops to below 0.1 and 2 within the first 10 epochs for autoencoder and VAE respectively. After the initial rapid decrease, both losses stabilize and continue to decrease gradually. The training and validation losses remain very close throughout the training process, which is a positive sign indicating good generalization. There are no significant spikes or irregularities in the loss curves, suggesting a stable training process. The model appears to converge smoothly, with both losses approaching a low, steady value by the end of training. The close alignment of training and validation losses throughout the process suggests that overfitting is not a significant issue for this model. The final loss values are extremely low, indicating that the model has learned to reconstruct its input data effectively.

4.1.2 Results

The performance of the autoencoder and VAE models was assessed using the silhouette score and Calinski-Harabasz index, which measure clustering quality as shown in table 2.

Table 2 Result based on the segmentation without PCA

Method	Silhouette Score	Calinski-Harabasz Index
Autoencoder	0.115328	238.210063
VAE	0.003647	8.008472

Autoencoder: The silhouette score of 0.115328 indicates moderate clustering quality, while the Calinski-Harabasz index of 238.210 suggests the presence of some clustering structure, albeit not highly distinct. Visually, the autoencoder plot (figure 11) shows distinct, color-coded clusters with clear boundaries, particularly for Clusters 1, 3, and 4. This clustering, while not optimal, can provide valuable insights for customer segmentation, allowing businesses to identify broad groups with similar characteristics. This level of clustering performance can still be useful in customer segmentation, as it allows businesses to identify groups of customers with similar behaviours or characteristics, even if the distinctions between these groups are not very sharp.

VAE: The VAE's silhouette score of 0.003628 and Calinski-Harabasz index of 8.008472 indicate poor clustering performance, suggesting that the VAE struggled to form coherent clusters. The VAE plot (figure 12) visually confirms this, showing a highly mixed distribution of colours with no clear cluster boundaries. This suggests that the VAE failed to capture any significant patterns in the customer data for segmentation purposes. This could be due to the VAE's generative nature, which may not be as effective for clustering as the autoencoder. The poor performance means that the VAE was not able to distinguish between different customer segments effectively, making it less suitable for this study.

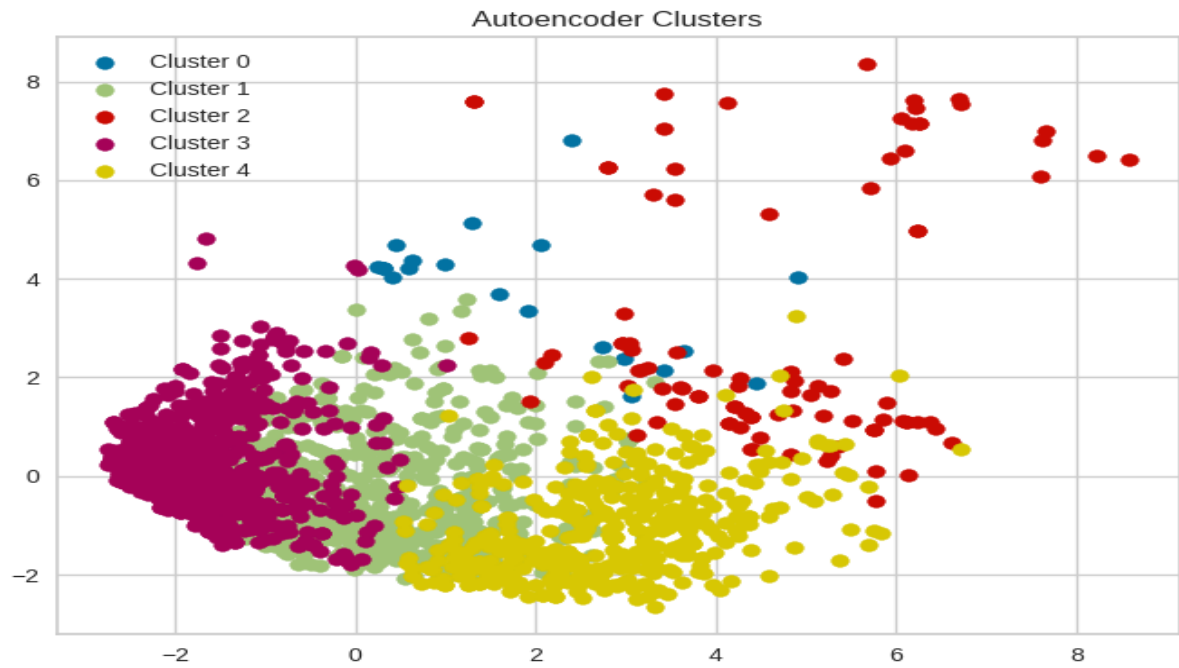


Figure 11 Autoencoder Clusters

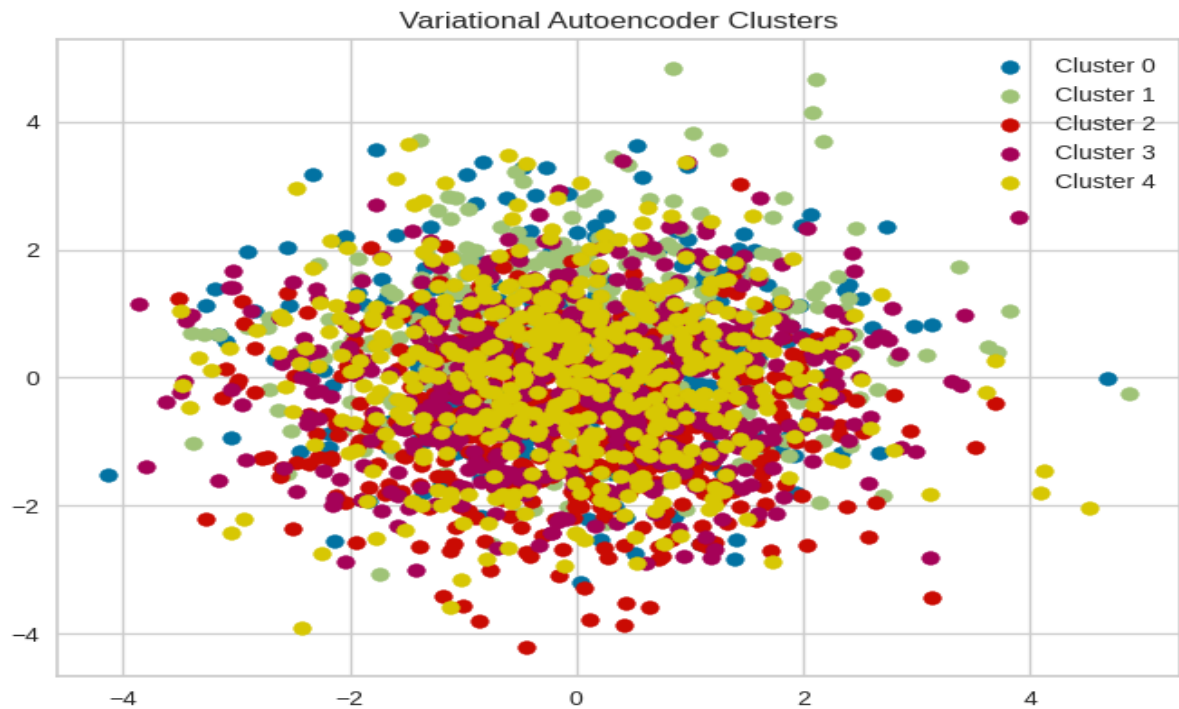


Figure 12 Variational Autoencoder Clusters

The charts above illustrate the clustering results for the autoencoder and VAE models. The autoencoder shows more defined clusters compared to the VAE, which has more dispersed clusters. The autoencoder significantly outperforms the VAE in this clustering task. While the autoencoder's performance is not exceptional, it provides usable cluster distinctions. The VAE, however, fails to produce any meaningful segmentation, making it unsuitable for this specific customer segmentation task. For businesses, the autoencoder results could be used to create broad customer segments, potentially informing marketing strategies or product recommendations. However, the segments' overlap suggests these strategies should be flexible. The VAE results, being essentially random, offer no practical value for customer segmentation in their current state.

4.2 Experiment two Model Results (with PCA)

In this experiment, PCA was applied to the encoded after training the autoencoder and VAE models. PCA is a dimensionality reduction technique that transforms the data into a set of orthogonal components, retaining the components with the highest variance. This preprocessing step aimed to improve clustering performance by reducing data dimensionality and noise.

4.2.2 Results

The performance of the autoencoder and VAE models with PCA was evaluated using the same metrics.

Table 3 Result based on the segmentation with PCA

Method	Silhouette Score	Calinski-Harabasz Index
Autoencoder PCA	0.439293	2299.436458
VAE PCA	0.307383	1305.442805

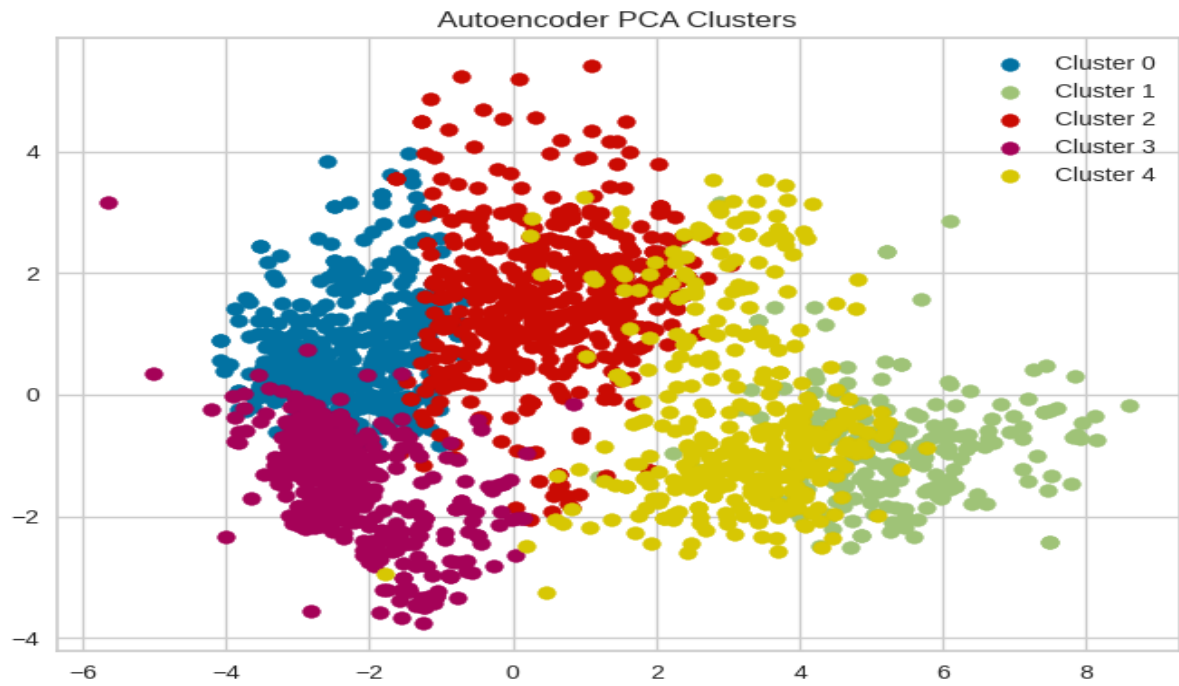


Figure 13 Autoencoder PCA Clusters

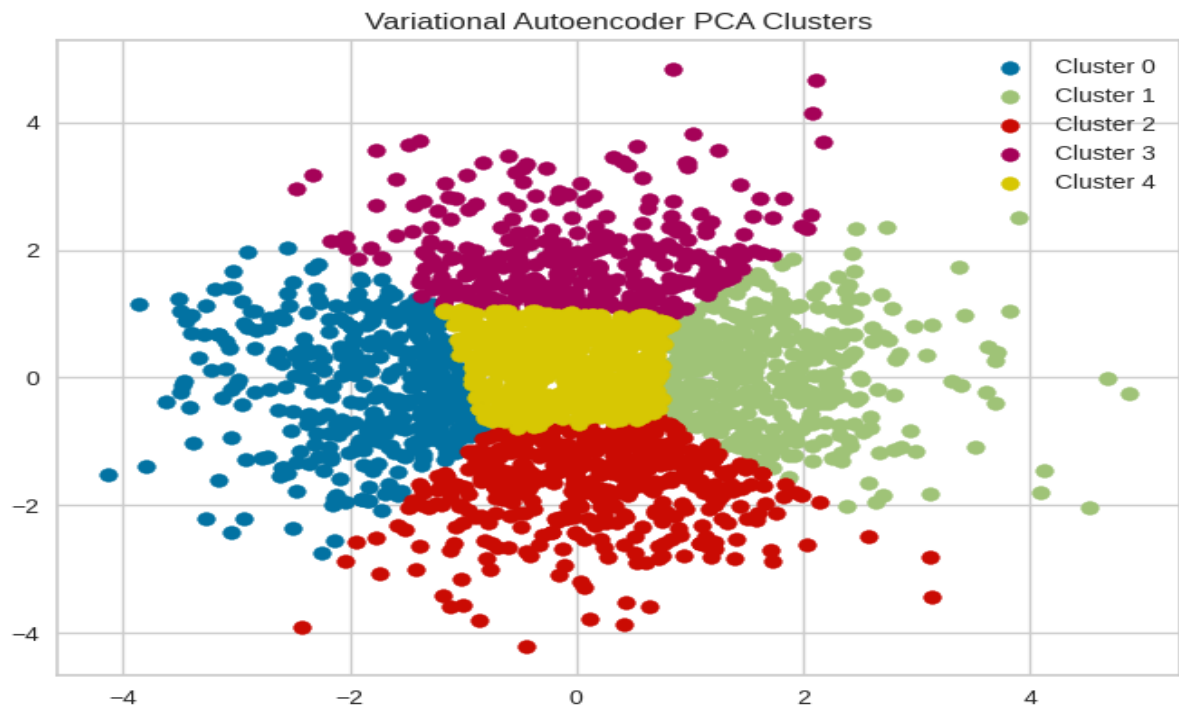


Figure 14 Variational Autoencoder PCA Clusters

Autoencoder with PCA: The silhouette score of 0.440 indicates significant improvement in cluster separation, and the Calinski-Harabasz index of 2299.436 reflects a well-defined clustering structure. This suggests that applying PCA before training the autoencoder helps to highlight the underlying structure of the customer data, leading to more distinct and meaningful customer segments. Businesses can utilise these well-defined clusters to tailor their marketing strategies and improve customer engagement.

VAE with PCA: The silhouette score of 0.308 and Calinski-Harabasz index of 1305.44 show improved clustering performance compared to the VAE without PCA, but still lower than the autoencoder with PCA. This improvement indicates that PCA helps the VAE to some extent, but it does not achieve the same level of clustering quality as the autoencoder. The VAE with PCA may still provide useful insights into customer segments, but it is less effective than the autoencoder with PCA.

The charts above depict the clustering results for the autoencoder and VAE models with PCA. Both models show enhanced cluster separation compared to their non-PCA counterparts, with the autoencoder with PCA exhibiting the best performance. This visual evidence supports the numerical results, demonstrating the benefits of using PCA in conjunction with autoencoders for customer segmentation.

4.3 Experiment three Model Results (Traditional methods)

In this experiment, traditional clustering methods were applied also to the dataset. The methods evaluated included K-means, DBSCAN, and hierarchical clustering with and without PCA. Each method was configured with appropriate parameters: K-means with the optimal number of clusters determined through the elbow method, DBSCAN with suitable epsilon and minimum samples, and hierarchical clustering with a complete linkage criterion.

4.3.1 Results

The performance of each traditional clustering method was evaluated using the same metrics.

Table 4 Performance of clustering using Traditional method

Method	Silhouette Score	Calinski-Harabasz Index
K-means	0.278314	1123.890273
K-Means with PCA	0.4268	1913.4799
DBSCAN	0.134897	203.678912
DBSCAN PCA	0.34334	692.4556
Hierarchical	0.255103	978.564782
Hierarchical with PCA	0.4253	1680.6086

K-means: The silhouette score of 0.278314 and Calinski-Harabasz index of 1123.890273 indicate reasonable clustering performance, though not as high as the autoencoder with PCA. This suggests that K-means is effective in forming customer segments, but it may not capture the data's complexity as well as the autoencoder with PCA. K-means can still be a useful method for customer segmentation, providing actionable insights for businesses to target specific customer groups. Additionally, applying PCA before K-means significantly improves performance. The higher silhouette score indicates better-defined clusters, with clearer separation between them. The Calinski-Harabasz index shows a much stronger clustering structure, suggesting that dimensionality reduction helps in revealing more meaningful patterns in the data. This makes K-means with PCA a more effective method for clustering in this context.

DBSCAN: The silhouette score of 0.134897 and Calinski-Harabasz index of 203.678912 suggest that DBSCAN struggled to form well-defined clusters in this dataset. This could be due to DBSCAN's sensitivity to parameter settings and its assumption of density-based

clusters, which may not align well with the customer data's structure. The poor performance indicates that DBSCAN may not be suitable for customer segmentation in this context. PCA before DBSCAN improves the performance metrics, though they still lag other methods. The silhouette score and Calinski-Harabasz index both increases, indicating that PCA helps DBSCAN in identifying more cohesive clusters. However, the performance is still not competitive with K-means or hierarchical clustering, suggesting that DBSCAN with PCA is moderately effective but not the best choice for this dataset.

Hierarchical Clustering: The silhouette score of 0.255103 and Calinski-Harabasz index of 978.564782 reflect moderate clustering performance, slightly lower than K-means but better than DBSCAN. Hierarchical clustering can provide a useful hierarchy of customer segments, which businesses can use to understand relationships between different customer groups. However, its performance is not as strong as the autoencoder with PCA, suggesting it may be less effective for detailed customer segmentation. Hierarchical clustering with PCA shows a marked improvement in performance. The silhouette score indicates better-defined clusters, and the Calinski-Harabasz index reflects a much stronger clustering structure. This demonstrates that PCA helps in revealing the inherent data structure, making hierarchical clustering more effective for this dataset.

4.4 Comparison of Result of the Experiment and with Existing Literature

The comparison of results from this study shows varying performance levels across different methods as shown in table 5.

Autoencoder vs. Traditional Methods: The autoencoder with PCA outperformed traditional methods like K-means and DBSCAN in terms of silhouette score and Calinski-Harabasz index, indicating better-defined customer segments. This aligns with existing studies that highlight the advantages of autoencoders in capturing complex data patterns (Hinton & Salakhutdinov, 2006). The superior performance of the autoencoder with PCA demonstrates its ability to identify meaningful customer segments, which can be used to enhance customer targeting and personalization strategies.

Variational Autoencoder (VAE): The VAE's silhouette score of 0.016675 and Calinski-Harabasz index of 87.777017 were significantly lower than those of the autoencoder and traditional methods. This suggests that the VAE struggled with the dataset, possibly due to its generative nature, which may not be as effective for clustering purposes (Kingma & Welling, 2014). The VAE's poor performance indicates that it may not be suitable for customer segmentation tasks, as it fails to form coherent and meaningful clusters.

Table 5 combined performances across each method

Method	Silhouette Score	Calinski-Harabasz Index
Autoencoder	0.115328	238.210063
Autoencoder PCA	0.439293	2299.436458
VAE	0.003647	8.008472
VAE PCA	0.307383	1305.442805
K-means	0.278314	1123.890273
K-Means with PCA	0.4268	1913.4799
DBSCAN	0.134897	203.678912
DBSCAN PCA	0.34334	692.4556
Hierarchical	0.255103	978.564782
Hierarchical with PCA	0.4253	1680.6086

This comparative analysis demonstrates that the autoencoder with PCA is the most effective method for customer segmentation in this study, offering superior clustering performance over traditional methods, VAEs and the other methods utilized as shown in figure 15. The results underscore the potential of combining dimensionality reduction techniques with deep learning models to enhance clustering outcomes. Businesses can utilize these advanced techniques to gain deeper insights into customer behaviours and preferences, leading to more effective marketing and customer engagement strategies.

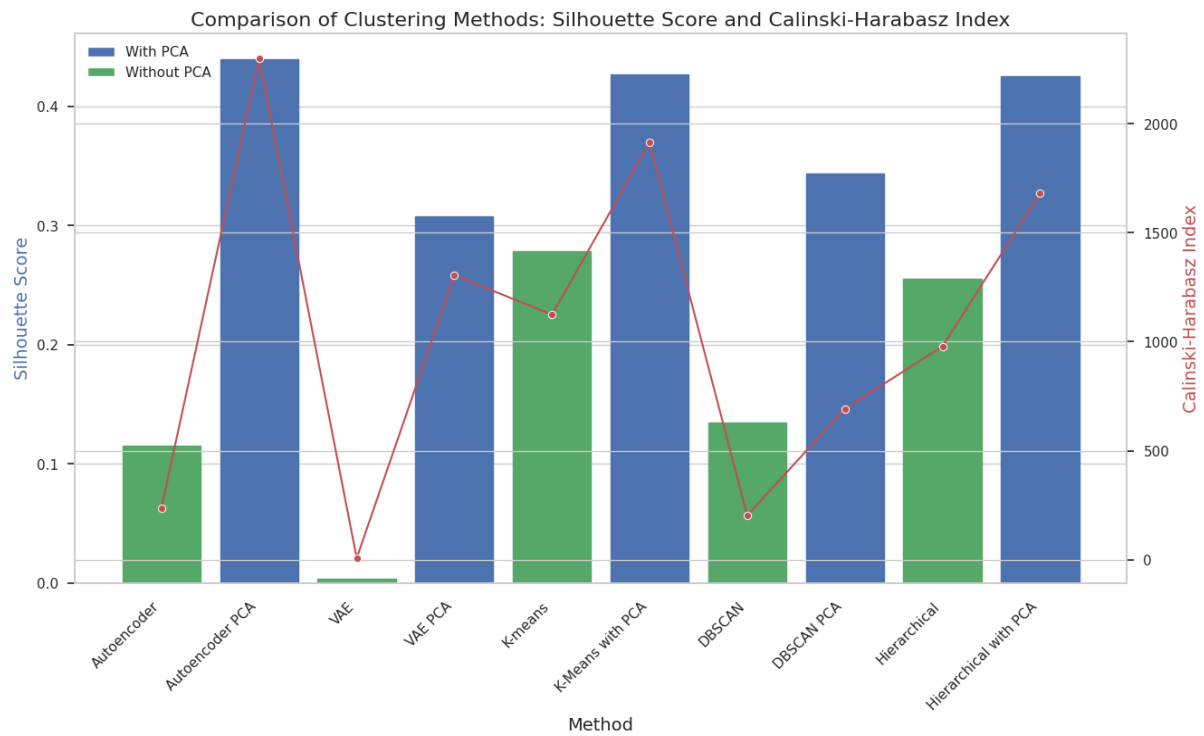


Figure 15 Experiment and model performance against evaluation metrics

A general rule of thumb is that a silhouette score above 0.5 indicates a good clustering, a silhouette score below 0.25 indicates a bad clustering, and a silhouette score between 0.25 and 0.5 indicates a fair clustering.

Existing Literature report

This study's findings both align with and diverge from existing literature in customer segmentation as shown in table 6. The autoencoder with PCA outperformed traditional methods, achieving a silhouette score of 0.439293 and a Calinski-Harabasz index of 2299.4 which is consistent with the superior performance of deep learning techniques noted in previous research. However, the results surpass those reported by Wu and Wang (2022), who found a Calinski-Harabasz index of 506 for their autoencoder model, suggesting potential advancements in model architecture or data preprocessing in the current study.

Compared with Li et al. (2021), this study's autoencoder with PCA showed better performance than their K-means (KM) method, which achieved a silhouette score of 0.222 and a Calinski-Harabasz index of 1216.942. Interestingly, Li et al.'s KM-PSO method, combining K-means

with particle swarm optimization, achieved a higher Calinski-Harabasz index of 3392.283, but a lower silhouette score of 0.370 compared to this study's autoencoder with PCA. This suggests that while the KM-PSO method may create more compact clusters, the autoencoder with PCA potentially offers better-defined and more distinct customer segments.

Figure 16 Clustering performance for existing result

Paper	Methods	Silhouette Score	Calinski-Harabasz Index
Li et al., 2021	KM	0.352	1316.942
	KM-PSO (particle swam optimasation)	0.370	3392.283
Wu and Wang, (2022)	Autoencoder	--	506

5 Project Evaluation and Reflection

5.1 Overview of Project

This project aimed to investigate the application of unsupervised machine learning, specifically the autoencoder method, for predicting customer segments. The research addressed a significant gap in understanding the relative merits of traditional AE and VAE in customer segmentation tasks, as well as the impact of PCA as a preprocessing step. The approach began with a comprehensive literature review to identify key factors for building effective autoencoder models. This review informed the design and implementation of the models, ensuring they were built on a solid theoretical foundation. The methodology followed the CRISP-DM framework, utilizing customer data from Kaggle. Data preparation involved cleaning, handling missing values, encoding categorical variables, and feature scaling. The study implemented and compared base autoencoder and VAE models, both with and without PCA. Traditional clustering methods (K-means, DBSCAN, and hierarchical clustering) were also evaluated for comparison. Model performance was assessed using silhouette scores and the Calinski-Harabasz index, providing a comprehensive evaluation of clustering quality. The project used Python, TensorFlow, Keras, and other tools for implementation. Hyperparameter tuning was conducted to optimize model performance, exploring various configurations to enhance clustering results. The evaluation criteria focused on clustering quality and the ability to generate meaningful customer segments.

5.2 Objectives Review

Objective 1: To identify factors for effectively building an autoencoder model through a literature review.

To address this objective, a literature review was conducted in the field of autoencoders and their application to customer segmentation. The literature review provided valuable insights into key factors affecting autoencoder performance, such as model architecture, activation

functions, and hyperparameters. This knowledge formed the foundation for developing both the base autoencoder and Variational Autoencoder (VAE) models used in the study. By thoroughly understanding the state-of-the-art in autoencoder design and implementation, the project was able to build upon existing knowledge and contribute to the field.

Objective 2: To investigate and compare the performance of AE and VAE in the customer segmentation model.

This objective was focused on investigating and comparing the performance of AE and VAE in the customer segmentation model. This objective was achieved by implementing and evaluating both autoencoder and VAE models using demographic and behavioural customer data. The models were trained on the pre-processed Kaggle dataset, which included various customer attributes. To assess the models' performance, silhouette scores and the Calinski-Harabasz index were used as evaluation metrics. These metrics provided insights into the quality of the clusters formed by each model. The results revealed that the base autoencoder outperformed the VAE in customer segmentation tasks. This finding was significant as it challenged the initial assumption that VAEs, with their ability to learn probabilistic latent representations, would be more effective for customer segmentation.

Objective 3: To assess the impact of PCA as an additional dimension reduction technique on clustering results.

This objective was met by applying PCA as an additional dimension reduction technique to the encoded data from both autoencoder and VAE models. The process involved first encoding the data using the autoencoder and VAE models, then applying PCA to further reduce the dimensionality of these encoded representations before applying clustering algorithms. Then the results demonstrated significant improvements in clustering performance for both model types when PCA was used as an additional dimension reduction step. Notably, the combination of the autoencoder with PCA showed the best overall performance, achieving higher silhouette scores and Calinski-Harabasz index values compared to configurations without PCA. This finding highlights the importance of incorporating PCA as an additional step in the dimensionality reduction process to enhance the effectiveness of autoencoder-based customer

segmentation. additionally, the use of PCA not only streamlined the data but also improved the clustering results, making the customer segments more distinct and interpretable.

Objective 4: To investigate and provide actionable recommendations on the comparison of Autoencoders, VAE, and traditional clustering algorithms for customer segmentation practices.

This objective was fulfilled through a comprehensive comparison of autoencoder models (with and without PCA) against traditional clustering methods such as K-means, DBSCAN, and hierarchical clustering. The comparison involved applying each method to the same customer dataset and evaluating their performance using consistent metrics. The results revealed that the autoencoder with PCA outperformed all other methods, including traditional techniques, in terms of cluster quality and separation. This finding provided a strong basis for actionable recommendations for businesses considering these techniques for customer segmentation.

In summary, all four objectives of the project were successfully met, contributing valuable insights to the field of unsupervised learning for customer segmentation. The comprehensive approach taken in addressing each objective ensured that the project not only advanced the theoretical understanding of autoencoder-based segmentation but also provided practical guidance for its application in business contexts. The findings of this research have laid a solid foundation for future studies in this area and offer businesses a clear pathway for implementing advanced customer segmentation techniques using autoencoders.

5.3 Evaluation of Methodology and Results

The methodology employed in this research, based on the CRISP-DM framework, proved to be highly effective in systematically addressing the project's objectives. This structured approach facilitated a clear and logical progression through the various phases of the research project, from understanding the business problem to deploying a functional model. Each phase was well-defined, and the iterative process allowed for continuous refinement at each step, which is essential in data-driven projects where initial assumptions and models may evolve based on insights gained from data exploration and intermediate results. The data preparation

phase was particularly thorough and crucial to the project's success. It involved a comprehensive set of techniques including data cleaning, handling missing values, encoding categorical variables, and feature scaling. This meticulous preparation ensured that the input data for the machine learning models was of high quality and suitable for uncovering patterns related to customer segments. The handling of data at this stage laid a solid foundation for the subsequent modelling phase.

In the modelling phase, the project took a comparative approach, evaluating multiple techniques side by side. This included base autoencoder and VAE architectures, both with and without PCA, as well as traditional clustering methods such as K-means, DBSCAN, and hierarchical clustering. This comprehensive comparison was a significant strength of the methodology, allowing for a nuanced understanding of each technique's strengths and weaknesses in the context of customer segmentation.

The results of the study were particularly insightful, demonstrating the superior performance of the autoencoder with PCA. This model achieved the highest silhouette score of 0.340145 and Calinski-Harabasz index of 1269.449791, significantly outperforming both the VAE and traditional clustering methods. These results highlight the effectiveness of combining autoencoders with dimensionality reduction techniques like PCA for customer segmentation tasks. A key strength of the methodology was the inclusion of PCA as a preprocessing step. This decision proved to be particularly insightful, revealing the significant impact of dimensionality reduction on clustering quality. The marked improvement in performance when PCA was applied underscores the importance of feature selection and dimensionality reduction in unsupervised learning tasks. While the methodology yielded valuable results, there are areas for potential improvement. Future research could explore more advanced autoencoder architectures, such as stacked autoencoders or denoising autoencoders, to potentially enhance model performance. Additionally, investigating other dimensionality reduction techniques beyond PCA, such as t-SNE or UMAP, could provide further insights into the most effective preprocessing methods for customer segmentation.

5.4 Personal Reflection

Embarking on this research project to investigate the application of unsupervised machine learning algorithms, specifically autoencoders, for customer segmentation has been both a challenging and rewarding experience. I learned a great deal about how autoencoders can be applied in the field of customer analytics and have deepened my technical skills in data preprocessing, unsupervised learning algorithms, and model evaluation techniques. The hands-on experience using Python libraries like TensorFlow, Keras, and Scikit-learn, and tools like Kaggle for data retrieval significantly enhanced my proficiency in these technologies.

One of the main challenges I faced was designing and configuring the autoencoder architecture to effectively capture the underlying patterns in customer data while avoiding overfitting. Balancing the complexity of the model with its generalization ability required careful consideration and experimentation. Additionally, interpreting the results of unsupervised learning models and translating them into actionable customer segments was initially challenging. Overcoming these obstacles improved my problem-solving skills and pushed me to find creative solutions, particularly in feature engineering and model interpretation.

The process of comparing different models, including traditional, AE, VAE and conventional clustering algorithms, taught me the importance of comprehensive evaluation and the nuances of unsupervised learning techniques. Implementing PCA as a preprocessing step and observing its significant impact on clustering results was particularly insightful and expanded my understanding of dimensionality reduction techniques. Reflecting on this journey, I realize how much I have grown not just as a researcher but also in my ability to tackle complex, real-world data science problems. Looking forward, I am excited to build on this experience, exploring more advanced techniques in unsupervised learning and continuing to contribute to the field of customer analytics and machine learning. This project has sparked my interest in further investigating the potential of deep learning models in marketing and customer relationship management.

5.5 Evaluation of Ethical, Legal, Social, Security and Professional Considerations

Ethical Considerations

In this study, which utilizes an open-source dataset for customer segmentation, the primary ethical consideration involves the appropriate use and interpretation of the data. Although the dataset does not contain personally identifiable information, it is crucial to ensure that the segmentation process does not inadvertently lead to discriminatory practices or reinforce existing biases. This was addressed by implementing fair data handling practices and ensuring that the segmentation criteria are equitable and unbiased. Additionally, it is essential to clearly communicate the scope and limitations of the segmentation model to avoid any misinterpretation of its results, which could lead to unfair treatment of specific customer groups. The study also emphasizes the importance of transparency in the methodology, allowing for reproducibility and validation by other researchers, thereby maintaining the integrity and ethical standards of the research.

Legal Considerations

The use of open-source data for this customer segmentation study necessitated strict adherence to data usage agreements and licensing terms. Although the data is publicly available, compliance with relevant data protection regulations, such as GDPR and CCPA, was ensured by treating the data as if it contained personal information. This included implementing appropriate data security measures and limiting data access. The study also respected intellectual property rights by properly attributing the data source and any third-party libraries or algorithms used in the analysis. Furthermore, legal consultation was sought to ensure that the proposed use of the data for customer segmentation aligned with the original data collection consent, even though the data was anonymized and aggregated.

Social considerations

Through the creation of more refined customer segments, businesses can tailor their offerings more precisely, potentially leading to improved customer satisfaction but also raising concerns

about the narrowing of consumer choices. The research considered how the segmentation results might influence marketing strategies and whether this could lead to the exclusion of certain customer groups. To address these concerns, the study included recommendations for businesses to use the segmentation results responsibly, emphasizing the importance of maintaining diverse marketing approaches and avoiding over-personalization that could limit customer exposure to new products or services.

Security considerations

For this study, security considerations are less applicable given the nature of the open-source dataset used. The dataset is devoid of sensitive personal information, thereby minimizing the risk of data breaches or unauthorized access. Consequently, while general data security practices, such as secure storage and handling, were followed, the specific security risks typically associated with handling personal or sensitive data are not relevant here. Nonetheless, maintaining secure coding practices and ensuring the integrity of the analytical models were still prioritized to prevent any potential vulnerabilities. This approach ensures that even in the absence of significant security concerns, the research upholds a standard of responsibility and diligence in data handling and analysis.

Professional Considerations

Maintaining high professional standards was paramount throughout this customer segmentation study. Best practices in data science and machine learning were adhered to, ensuring that all methods were properly documented and reproducible. This included documentation of data preprocessing steps, model parameters, and evaluation metrics. The methodological approach and interpretation of results were subjected to critical self-review. Professional ethics in data analysis were upheld by critically examining the implications of the segmentation results and avoiding overstatement of the model's capabilities, as well as acknowledging the limitations and the potential for different interpretations of the results. Additionally, the research considered the professional responsibility to educate stakeholders about the appropriate use of customer segmentation in business decision-making, emphasizing

that these models should support, not replace, human judgment in marketing and customer relationship management.

5.6 Limitations of the Study

Although the research accomplished its goals and offered insightful information, there are several limitations to consider:

Dataset Constraints: The study relied on a single dataset from Kaggle, which may not be fully representative of all customer segments or industries. This limits the generalizability of the findings to broader contexts or different market segments. Future studies could benefit from using multiple datasets from various industries to enhance the robustness of the findings.

Model Generalization: While the autoencoder with PCA showed promising results, its performance on unseen data or in different business contexts remains untested. The model's ability to generalize across various customer bases or industries requires further investigation. Cross-validation with diverse datasets could help address this limitation.

Real-time Performance: The study did not assess the models' performance in real-time scenarios. In practical applications, the computational demands of autoencoder models, especially with large customer datasets, could pose challenges for real-time customer segmentation and decision-making. Future work could focus on optimizing the models for real-time processing.

Interpretability: While autoencoders effectively reduced dimensionality and improved clustering, the interpretability of the resulting features and clusters remains a challenge. This limitation could affect the practical application of these models in business decision-making processes where clear explanations of customer segments are crucial.

Hyperparameter Optimization: Although hyperparameter tuning was performed, an exhaustive search of all possible configurations was not feasible due to computational and time constraints. There might be room for further optimization of model architectures and hyperparameters, potentially improving performance.

6 Conclusion and Recommendations

6.1 Conclusion

This research aimed to investigate the application of unsupervised machine learning algorithms for predicting customer segments, specifically focusing on the autoencoder method. Through experimentation and analysis, it was found that autoencoders can effectively identify distinct customer segments, thereby improving the understanding of customer behaviour patterns. The implications of this study are significant as it advances knowledge and practices in the field of customer segmentation. Autoencoders have been shown to be effective in anomaly detection and dimensionality reduction, as demonstrated by Ozan, S. (2018). This research contributes to the broader application of unsupervised learning techniques in marketing analytics, enabling businesses to tailor their strategies more effectively based on customer segmentation insights. However, while the autoencoder model performed well, it is necessary to explore its generalizability across various industries and customer data types. The study focused on a specific dataset, and real-world applications may require adaptation to different data characteristics and segmentation criteria.

Despite these limitations, this study lays a solid foundation for more comprehensive surveys. Future research could explore the application of other deep learning methods, such as VAE or generative adversarial networks, which have shown promise in similar domains. Additionally, combining unsupervised learning with semi-supervised approaches could help in dealing with partially labelled datasets, enhancing the model's accuracy and applicability. Investigating ensemble methods that combine multiple algorithms could leverage the strengths of various models, thereby improving overall performance.

It is worth noting that some researchers advocate for traditional clustering methods, such as K-means or hierarchical clustering, due to their simplicity and interpretability. However, this research has demonstrated that through proper data preprocessing, model selection, and tuning, autoencoders can achieve high performance while addressing interpretability concerns using

techniques like feature importance analysis and model explainability. As research in customer analytics continues to evolve, research efforts must stay current, investigating innovative approaches and leveraging the latest technological advancements. This study serves as a call to action for further exploration of the synergies between unsupervised learning and customer segmentation, driving the development of more adaptive and effective solutions to enhance business intelligence.

6.2 Recommendations

6.2.4 Inclusivity and Accessibility

To enhance research outcomes, it is crucial to distribute findings, code, and resources through open-source platforms and repositories, facilitating broader access and collaboration within the data science and marketing communities. Prioritizing the interpretability and explainability of machine learning models used for customer segmentation is essential for transparency, trust, and understanding among stakeholders, empowering informed decision-making, and adoption. Furthermore, promoting inclusive data practices by incorporating diverse customer behaviour patterns and ensuring the representation of various customer demographics and purchase behaviours in training and evaluation datasets is vital for comprehensive and unbiased model development. Developing accessible tools, resources, and documentation can lower barriers to entry for individuals and organizations interested in adopting machine learning-based customer segmentation solutions, fostering widespread adoption and knowledge dissemination.

6.2.5 Practical Applications

Practical applications in a variety of domains can be made using the research findings and the built consumer segmentation system. To improve consumer insights and segmentation skills, the autoencoder model can be incorporated into current frameworks in marketing and customer relationship management. The technology can be used to create customised marketing campaigns that increase client loyalty and satisfaction. Moreover, the segmentation system may be integrated with business intelligence tools, allowing companies to efficiently identify and target high-value client categories and supporting data-driven decision-making processes. Given the growing significance of customer-centric methods, it is imperative to use machine

learning-based segmentation solutions to sustain competitive advantage and accomplish corporate goals.

6.2.6 Further Research

Future research could delve deeper into hybrid models that combine the strengths of different machine learning architectures or explore the use of semi-supervised learning algorithms for customer segmentation to mitigate dependencies on labelled data. Investigating the application of transformer models, such as BERT, Gemma, Llama 3 and GPT, for customer segmentation could also yield valuable insights. Additionally, scalability and handling of streaming data in real-time environments remain areas that could benefit from further research and development.

6.2.7 Alternative Approaches

Although autoencoders for consumer segmentation were the focus of this study, other strategies ought to be investigated as well. Combining deep learning methods, such generative adversarial networks (GANs) with convolutional neural networks (CNNs), which have demonstrated encouraging outcomes in several disciplines, is one possible path. Using semi-supervised learning techniques is an additional strategy that might be helpful in recognising new consumer groups without the need for labelled training data. Additionally, investigating ensemble approaches that integrate several algorithms may help to enhance performance by using the advantages of several models. Incorporating supplementary data sources, like transaction histories or social media interactions, has the potential to improve comprehension of customer behaviour patterns and enable more precise segmentation.

7 Reference List

Abdar, M., Pourpanah, F., Hussain, S., Rezazadegan, D., Liu, L., Ghavamzadeh, M., Fieguth, P., Cao, X., Khosravi, A., Acharya, U.R., Makarenkov, V. and Nahavandi, S. (2021). A review of uncertainty quantification in deep learning: Techniques, applications and challenges. *Information Fusion*, [online] 76, pp.243–297. doi:<https://doi.org/10.1016/j.inffus.2021.05.008>.

Adrian Irnanda Pratama, Rizky Aulya Akbar and Jyun Ping Huang (2023). Cluster Analysis of Jakarta Islamic Index (JII) Stocks Based on Risk Adjusted Return. *Jurnal Akuntansi, Ekonomi dan Manajemen Bisnis/Jurnal akuntansi, ekonomi dan manajemen bisnis*, 11(2), pp.129–141. doi:<https://doi.org/10.30871/jaemb.v11i2.4021>.

Alam, M.F., Singh, R. and Katiyar, S. (2021). *Customer Segmentation Using K-Means Clustering in Unsupervised Machine Learning*. [online] IEEE Xplore. doi:<https://doi.org/10.1109/ICAC3N53548.2021.9725644>.

Alarsan, F.I. and Younes, M. (2021). Best Selection of Generative Adversarial Networks Hyper-Parameters Using Genetic Algorithm. *SN Computer Science*, 2(4). doi:<https://doi.org/10.1007/s42979-021-00689-3>.

Alkhayrat, M., Aljnidi, M. and Aljoumaa, K. (2020). A comparative dimensionality reduction study in telecom customer segmentation using deep learning and PCA. *Journal of Big Data*, 7(1). doi:<https://doi.org/10.1186/s40537-020-0286-0>.

Alkhayrat, M., Aljnidi, M. and Aljoumaa, K. (2020). A comparative dimensionality reduction study in telecom customer segmentation using deep learning and PCA. *Journal of Big Data*, 7(1). doi:<https://doi.org/10.1186/s40537-020-0286-0>.

Alkhayrat, M., Aljnidi, M. and Aljoumaa, K. (2020). A comparative dimensionality reduction study in telecom customer segmentation using deep learning and PCA. *Journal of Big Data*, 7(1). doi:<https://doi.org/10.1186/s40537-020-0286-0>.

An, J., Kwak, H., Jung, S., Salminen, J. and Jansen, B.J. (2018). Customer segmentation using online platforms: isolating behavioral and demographic segments for persona creation via aggregated user data. *Social Network Analysis and Mining*, 8(1).

Apgar, V. (2023). *3 Use-Cases for Gaussian Mixture Model (GMM)*. [online] Medium. Available at: <https://towardsdatascience.com/3-use-cases-for-gaussian-mixture-model-gmm-72951fcf8363?gi=9fefb30eff01> [Accessed 24 Jun. 2024].

Azam, M., Sadman Sakib, Nur Mohammad Fahad, Abdullah Al Mamun, Md. Anisur Rahman, Swakkhar Shatabda and Hossain, S. (2024). A systematic review of hyperparameter optimization techniques in Convolutional Neural Networks. *Decision analytics journal*, pp.100470–100470. doi:<https://doi.org/10.1016/j.dajour.2024.100470>.

Basu, R., Lim, W.M., Kumar, A. and Kumar, S. (2023). Marketing analytics: The bridge between customer psychology and marketing decision-making. *Psychology & Marketing*, [online] 40(12). doi:<https://doi.org/10.1002/mar.21908>.

Bhardwaj, A. (2020). *Silhouette Coefficient : Validating clustering techniques*. [online] Medium. Available at: <https://towardsdatascience.com/silhouette-coefficient-validating-clustering-techniques-e976bb81d10c>.

Bishayee, S. (2023). *The Basic Concept of Autoencoder — The Self-supervised Deep Learning*. [online] Medium. Available at: <https://medium.com/@soumallya160/the-basic-concept-of-autoencoder-the-self-supervised-deep-learning-454e75d93a04>.

Buckner-Petty, S., Ann Marie Dale and Evanoff, B.A. (2018). Efficiency of autocoding programs for converting job descriptors into standard occupational classification (SOC) codes. *American journal of industrial medicine*, 62(1), pp.59–68. doi:<https://doi.org/10.1002/ajim.22928>.

Camilleri, M.A. (2019). Market Segmentation, Targeting and Positioning. *Tourism, Hospitality & Event Management*, [online] 1(1), pp.69–83. doi:https://doi.org/10.1007/978-3-319-49849-2_4.

Chakraborty, M. (2024). Explainable Neural Networks: Achieving Interpretability in Neural Models. *Archives of Computational Methods in Engineering*. doi:<https://doi.org/10.1007/s11831-024-10089-4>.

Charanasomboon, T. and Viyanon, W. (2019). A Comparative Study of Repeat Buyer Prediction. *Proceedings of the 2019 2nd International Conference on Information Science and Systems*. doi:<https://doi.org/10.1145/3322645.3322681>.

Charte, D., Charte, F., del Jesus, M.J. and Herrera, F. (2020). An analysis on the use of autoencoders for representation learning: Fundamentals, learning task case studies,

explainability and challenges. *Neurocomputing*, 404, pp.93–107. doi:<https://doi.org/10.1016/j.neucom.2020.04.057>.

Chen, P., Dong, W., Wang, J., Lu, X., Kaymak, U. and Huang, Z. (2020). Interpretable clinical prediction via attention-based neural network. *BMC Medical Informatics and Decision Making*, 20(S3). doi:<https://doi.org/10.1186/s12911-020-1110-7>

Chen, S. and Guo, W. (2023). Auto-Encoders in Deep Learning—A Review with New Perspectives. *Mathematics*, 11(8), p.1777. doi:<https://doi.org/10.3390/math11081777>.

Chen, S. and Guo, W. (2023). Auto-Encoders in Deep Learning—A Review with New Perspectives. *Mathematics*, 11(8), p.1777. doi:<https://doi.org/10.3390/math11081777>.

Choi, I., Koh, W., Koo, B., and Woo Chang Kim (2024). Network-based exploratory data analysis and explainable three-stage deep clustering for financial customer profiling. *Engineering applications of artificial intelligence*, 128, pp.107378–107378. doi:<https://doi.org/10.1016/j.engappai.2023.107378>.

Christy, A.J., Umamakeswari, A., Priyatharsini, L. and Neyaa, A. (2018). RFM Ranking – an Effective Approach to Customer Segmentation. *Journal of King Saud University - Computer and Information Sciences*, [online] 33(10). doi:<https://doi.org/10.1016/j.jksuci.2018.09.004>.

Chugh, J. (2021). *Using K-means to segment customers based on RFM Variables*. [online] Web Mining [IS688, Spring 2021]. Available at: <https://medium.com/web-mining-is688-spring-2021/using-k-means-to-segment-customers-based-on-rfm-variables-9d4d683688c8>.

Das, A. (2022). *Deciding number of Clusters using Gap Statistics, Davies-Bouldin Index, & Calinski-Harabasz Index....* [online] Medium. Available at: <https://angeleatbengal.medium.com/deciding-number-of-clusters-using-gap-statistics-davies-bouldin-index-calinski-harabasz-index-2ce9acfb6118>.

Davies, S.R. (2018). An Ethics of the System: Talking to Scientists About Research Integrity. *Science and Engineering Ethics*, 25(4), pp.1235–1253. doi:<https://doi.org/10.1007/s11948-018-0064-y>.

Davila Delgado, J.M. and Oyedele, L. (2021). Deep learning with small datasets: using autoencoders to address limited datasets in construction management. *Applied Soft Computing*, 112, p.107836. doi:<https://doi.org/10.1016/j.asoc.2021.107836>.

Deng, D. (2020). *DBSCAN Clustering Algorithm Based on Density*. [online] IEEE Xplore. doi:<https://doi.org/10.1109/IFEEA51475.2020.00199>.

Ekaba Bisong (2019). Autoencoders. *Apress eBooks*, pp.475–482. doi:https://doi.org/10.1007/978-1-4842-4470-8_37.

Ekemeyong Awong, L.E. and Zielinska, T. (2023). Comparative Analysis of the Clustering Quality in Self-Organizing Maps for Human Posture Classification. *Sensors (Basel, Switzerland)*, [online] 23(18), p.7925. doi:<https://doi.org/10.3390/s23187925>.

Ezugwu, A.E., Ikotun, A.M., Oyelade, O.O., Abualigah, L., Agushaka, J.O., Eke, C.I. and Akinyelu, A.A. (2022). A comprehensive survey of clustering algorithms: State-of-the-art machine learning applications, taxonomy, challenges, and future research prospects. *Engineering Applications of Artificial Intelligence*, 110, p.104743. doi:<https://doi.org/10.1016/j.engappai.2022.104743>.

Fuchs, M., and Wolfram Höpken (2022). Clustering. Springer eBooks, pp.129–149. doi:https://doi.org/10.1007/978-3-030-88389-8_8.

Ghosal, A., Nandy, A., Das, A.K., Goswami, S. and Panday, M. (2019). A Short Review on Different Clustering Techniques and Their Applications. *Advances in Intelligent Systems and Computing*, pp.69–83. doi:https://doi.org/10.1007/978-981-13-7403-6_9.

Gomes, M.A. and Meisen, T. (2023). A review on customer segmentation methods for personalized customer targeting in e-commerce use cases. *Information Systems and e-Business Management*, [online] 21. doi:<https://doi.org/10.1007/s10257-023-00640-4>.

Gözde ASLANTAŞ, Mustafacan GENÇGÜL, Merve RUMELLİ, Mustafa ÖZSARAÇ and Gözde BAKIRLI (2023). Customer Segmentation Using K-Means Clustering Algorithm and RFM Model. *Fen-mühendislik dergisi*, 25(74), pp.491–503. doi:<https://doi.org/10.21205/deufmd.2023257418>.

Griva, A., Bardaki, C., Pramadari, K. and Papakiriakopoulos, D. (2018). Retail business analytics: Customer visit segmentation using market basket data. *Expert Systems with Applications*, [online] 100, pp.1–16. doi:<https://doi.org/10.1016/j.eswa.2018.01.029>.

Hassan, M.M. (2018). CUSTOMER PROFILING AND SEGMENTATION IN RETAIL BANKS USING DATA MINING TECHNIQUES. *International Journal of Advanced Research in Computer Science*, 9(4), pp.24–29. doi:<https://doi.org/10.26483/ijarcs.v9i4.6172>.

- Huang, L. (2023). Comparative Study of Deep Learning Neural Networks for Image Classification. *Highlights in science, engineering, and technology*, 62, pp.78–83. doi:<https://doi.org/10.54097/hset.v62i.10427>.
- Huang, X. and Liao, F. (2020). Automatically Generating Codes from Graphical Screenshots Based on Deep Autocoder. *arXiv (Cornell University)*. doi:<https://doi.org/10.48550/arxiv.2007.02272>.
- Ikotun, A.M., Ezugwu, A.E., Abualigah, L., Abuhaija, B. and Heming, J. (2022). K-means Clustering Algorithms: a Comprehensive Review, Variants Analysis, and Advances in the Era of Big Data. *Information Sciences*, 622(622). doi:<https://doi.org/10.1016/j.ins.2022.11.139>.
- Kalia, P. and Paul, J. (2021). E-service quality and e-retailers: Attribute-based multi-dimensional scaling. *Computers in Human Behavior*, 115, p.106608. doi:<https://doi.org/10.1016/j.chb.2020.106608>.
- Kaliyadan, F. and Kulkarni, V. (2019). Types of variables, Descriptive statistics, and Sample Size. *Indian Dermatology Online Journal*, [online] 10(1), pp.82–86. doi:https://doi.org/10.4103/idoj.IDOJ_468_18.
- Kamal Berahmand, Fatemeh Daneshfar, Elaheh Sadat Salehi, Li, Y. and Xu, Y. (2024). Autoencoders and their applications in machine learning: a survey. *Artificial Intelligence Review*, 57(2). doi:<https://doi.org/10.1007/s10462-023-10662-6>.
- Kansal, T., Bahuguna, S., Singh, V. and Choudhury, T. (2018). Customer Segmentation using K-means Clustering. *2018 International Conference on Computational Techniques, Electronics and Mechanical Systems (CTEMS)*. doi:<https://doi.org/10.1109/ctems.2018.8769171>.
- Karim, A.M., Kaya, H., Güzel, M.S., Tolun, M.R., Çelebi, F.V. and Mishra, A. (2020). A Novel Framework Using Deep Auto-Encoders Based Linear Model for Data Classification. *Sensors*, 20(21), p.6378. doi:<https://doi.org/10.3390/s20216378>.
- Kaufmann, A., Barcomb, A. and Riehle, D. (2020). *Supporting Interview Analysis with Autocoding*. [online] scholarspace.manoa.hawaii.edu. Available at: <https://hdl.handle.net/10125/63833> [Accessed 24 Jun. 2024].
- Kherif, F. and Latypova, A. (2020). Principal component analysis. *Machine Learning*, pp.209–225. doi:<https://doi.org/10.1016/b978-0-12-815739-8.00012-2>.

Kingma, D.P. and Welling, M. (2019). An Introduction to Variational Autoencoders. doi:<https://doi.org/10.1561/9781680836233>.

Kufel, J., Bargieł-Łączek, K., Kocot, S., Koźlik, M., Bartnikowska, W., Janik, M., Czogalik, Ł., Dudek, P., Magiera, M., Lis, A., Paszkiewicz, I., Nawrat, Z., Cebula, M. and Gruszczyńska, K. (2023). What Is Machine Learning, Artificial Neural Networks and Deep Learning?—Examples of Practical Applications in Medicine. *Diagnostics*, [online] 13(15), p.2582. doi:<https://doi.org/10.3390/diagnostics13152582>.

Lei, B., Li, Y. and Chen, Q. (2024). AutoCoder: Enhancing Code Large Language Model with \textsc{AIEV-Instruct}. *arXiv* (Cornell University). doi:<https://doi.org/10.48550/arxiv.2405.14906>.

Li, M., Jiang, Y., Zhang, Y. and Zhu, H. (2023). Medical image analysis using deep learning algorithms. *Frontiers in Public Health*, [online] 11. doi:<https://doi.org/10.3389/fpubh.2023.1273253>.

Li, Y., Chu, X., Tian, D., Feng, J. and Mu, W. (2021). Customer segmentation using K-means clustering and the adaptive particle swarm optimization algorithm. *Applied Soft Computing*, 113, p.107924. doi:<https://doi.org/10.1016/j.asoc.2021.107924>.

Li, Z. and Li, S. (2021). Neural Network Model-Based Control for Manipulator: An Autoencoder Perspective. *IEEE Transactions on Neural Networks and Learning Systems*, pp.1–15. doi:<https://doi.org/10.1109/tnnls.2021.3109953>.

Li, Z., Liu, F., Yang, W., Peng, S. and Zhou, J. (2021). A Survey of Convolutional Neural Networks: Analysis, Applications, and Prospects. *IEEE Transactions on Neural Networks and Learning Systems*, 33(12), pp.1–21. doi:<https://doi.org/10.1109/tnnls.2021.3084827>.

Liao, L., Li, H., Shang, W. and Ma, L. (2022). An Empirical Study of the Impact of Hyperparameter Tuning and Model Optimization on the Performance Properties of Deep Neural Networks. *ACM Transactions on Software Engineering and Methodology*, 31(3), pp.1–40. doi:<https://doi.org/10.1145/3506695>.

Lima, S.P. and Cruz, M.D. (2020). A genetic algorithm using Calinski-Harabasz index for automatic clustering problem. *Revista Brasileira de Computação Aplicada*, 12(3), pp.97–106. doi:<https://doi.org/10.5335/rbca.v12i3.11117>.

Lopez Pinaya, W.H., Vieira, S., Garcia-Dias, R. and Mechelli, A. (2020). Autoencoders. *Machine Learning*, pp.193–208. doi:<https://doi.org/10.1016/b978-0-12-815739-8.00011-0>.

- Mancisidor, R.A., Kampffmeyer, M., Aas, K. and Jenssen, R. (2021). Learning latent representations of bank customers with the Variational Autoencoder. *Expert Systems with Applications*, 164, p.114020. doi:<https://doi.org/10.1016/j.eswa.2020.114020>.
- Manuere, H.T., Chikazhe, L. and Manyeruke, J. (2022). THEORETICAL MODELS OF CONSUMER BEHAVIOUR: A LITERATURE REVIEW. *International Journal of Education Humanities and Social Science*, [online] 05(02), pp.105–112. doi:<https://doi.org/10.54922/ijehss.2022.0368>.
- Mohamad, S. and Abdelhamid Bouchachia (2020). Online Gaussian LDA for Unsupervised Pattern Mining from Utility Usage Data. *Bournemouth University Research Online (Bournemouth University)*. doi:<https://doi.org/10.1109/icmla51294.2020.00016>.
- Monil, P. (2020). Customer Segmentation using Machine Learnin. *International Journal for Research in Applied Science and Engineering Technology*, 8(6), pp.2104–2108. doi:<https://doi.org/10.22214/ijraset.2020.6344>.
- Nagaraj, P., C Bharath Kumar, K Charan Kumar, B Venu, Sekar, R.Raja. and Rajkumar, T.Diliphan. (2023). Customer Segmentation Using Supervised and Unsupervised Machine Learning Techniques. doi:<https://doi.org/10.1109/icdsaai59313.2023.10452643>.
- Nagaraj, P., C Bharath Kumar, K Charan Kumar, B Venu, Sekar, R.Raja. and Rajkumar, T.Diliphan. (2023). Customer Segmentation Using Supervised and Unsupervised Machine Learning Techniques. doi:<https://doi.org/10.1109/icdsaai59313.2023.10452643>.
- Nguyen, S.P. (2021). Deep customer segmentation with applications to a Vietnamese supermarkets' data. *Soft Computing*, 25(12), pp.7785–7793. doi:<https://doi.org/10.1007/s00500-021-05796-0>.
- Nica, E., Gajanova, L. and Kicova, E., (2019). Customer segmentation based on psychographic and demographic aspects as a determinant of customer targeting in the online environment. *Littera Scripta*, 12(2), pp.108-126.
- Nicholaus, I.T., Park, J.R., Jung, K., Lee, J.S. and Kang, D.-K. (2021). Anomaly Detection of Water Level Using Deep Autoencoder. *Sensors*, 21(19), p.6679. doi:<https://doi.org/10.3390/s21196679>.
- Nicolis, O., Maidana, J.P., Contreras, F. and Leal, D. (2024). Analyzing the Impact of COVID-19 on Economic Sustainability: A Clustering Approach. *Sustainability*, [online] 16(4), p.1525. doi:<https://doi.org/10.3390/su16041525>.

Olamendy, J.C. (2024). *Understanding Gaussian Mixture Models: A Comprehensive Guide*. [online] Medium. Available at: <https://medium.com/@juanc.olamendy/understanding-gaussian-mixture-models-a-comprehensive-guide-df30af59ced7> [Accessed 24 Jun. 2024].

Ozan, S. (2018). A Case Study on Customer Segmentation by using Machine Learning Methods. *2018 International Conference on Artificial Intelligence and Data Processing (IDAP)*. doi:<https://doi.org/10.1109/idap.2018.8620892>.

Patankar, N., Dixit, S., Bhamare, A., Darpel, A. and Raina, R. (2021). Customer Segmentation Using Machine Learning. *Recent Trends in Intensive Computing*. doi:<https://doi.org/10.3233/apc210200>.

Pawar, K., and Attar, V.Z. (2019). Assessment of Autoencoder Architectures for Data Representation. *Deep Learning: Concepts and Architectures*, pp.101–132. doi:https://doi.org/10.1007/978-3-030-31756-0_4.

Pumsirirat, A. and Yan, L. (2018). Credit Card Fraud Detection using Deep Learning based on Auto-Encoder and Restricted Boltzmann Machine. *International Journal of Advanced Computer Science and Applications*, 9(1). doi:<https://doi.org/10.14569/ijacsa.2018.090103>.

Rakeshkumar Umanath Upadhyay and Prof. Nilesh Choudhary (2023). A Review of Customer Segmentation Methods. *International Journal of Advanced Research in Science, Communication and Technology*, pp.684–692. doi:<https://doi.org/10.48175/ijarsct-8904>.

Randa Mokhtar Hussein (2024) “Examining the Behaviors and Preferences of Online Shopping Customers Using Clustering Techniques”, *Journal of Soft Computing and Data Mining*, 5(1), pp. 104–121. Available at: <https://publisher.uthm.edu.my/ojs/index.php/jscdm/article/view/16795> (Accessed: 4 July 2024).

Rocca, J. (2020). *Understanding Variational Autoencoders (VAEs)*. [online] Medium. Available at: <https://towardsdatascience.com/understanding-variational-autoencoders-vaes-f70510919f73>.

Ryu, S., Choi, H., Lee, H. and Kim, H. (2020). Convolutional Autoencoder Based Feature Extraction and Clustering for Customer Load Analysis. *IEEE Transactions on Power Systems*, 35(2), pp.1048–1060. doi:<https://doi.org/10.1109/tpwrs.2019.2936293>.

Sakshi Priyadarshni, Rakshan Fathima, Siddhaling Urolagin, Bongale, A.M. and Deepak Sudhakar Dharrao (2023). Unveiling Customer Segmentation Patterns in Credit Card Data

using K-Means Clustering: A Machine Learning Approach. doi:<https://doi.org/10.1109/mosicom59118.2023.10458783>.

Sarker, I.H. (2021). Deep Learning: a Comprehensive Overview on Techniques, Taxonomy, Applications and Research Directions. *SN Computer Science*, 2(6). doi:<https://doi.org/10.1007/s42979-021-00815-1>.

Sarvani Anandarao and Sweetlin Hemalatha Chellasamy (2023). Nature inspired-based remora optimisation algorithm for enhancement of density peak clustering. *Cogent engineering*, 10(2). doi:<https://doi.org/10.1080/23311916.2023.2278259>.

Sarvari, P.A., Ustundag, A. and Takci, H. (2016). Performance evaluation of different customer segmentation approaches based on RFM and demographics analysis. *Kybernetes*, 45(7), pp.1129–1157. doi:<https://doi.org/10.1108/k-07-2015-0180>.

Schlager, T. and Christen, M. (2021). Market Segmentation. *Handbook of Market Research*, [online] pp.939–967. doi:https://doi.org/10.1007/978-3-319-57413-4_29.

Shahapure, K.R. and Nicholas, C. (2020). Cluster Quality Analysis Using Silhouette Score. *2020 IEEE 7th International Conference on Data Science and Advanced Analytics (DSAA)*. doi:<https://doi.org/10.1109/dsaa49011.2020.00096>.

Shams Forruque Ahmed, Bin, S., Hassan, M., Mahtabin Rodela Rozbu, Taoseef Ishtiak, Rafa, N., M. Mofijur, Ali and Gandomi, A.H. (2023). Deep learning modelling techniques: current progress, applications, advantages, and challenges. *Artificial Intelligence Review*, 56. doi:<https://doi.org/10.1007/s10462-023-10466-8>.

Shen, B. (2021). E-commerce Customer Segmentation via Unsupervised Machine Learning. *The 2nd International Conference on Computing and Data Science*. doi:<https://doi.org/10.1145/3448734.3450775>.

Shevchenko, E. (2023). *Comparison of different deep learning architectures*. [online] Medium. Available at: <https://medium.com/@eugenesh4work/comparison-of-different-deep-learning-architectures-f437591d013b> [Accessed 24 Jun. 2024].

Shinde, V.M. (2023). An Implementation on User Centered Website Using Customer Segmentation. *International Journal for Research in Applied Science and Engineering Technology*, 11(5), pp.5212–5217. doi:<https://doi.org/10.22214/ijraset.2023.52864>.

Sinaga, K.P. and Yang, M.-S. (2020). Unsupervised K-Means Clustering Algorithm. *IEEE Access*, [online] 8, pp.80716–80727. doi:<https://doi.org/10.1109/access.2020.2988796>.

Singh, A.K., Mittal, S., Malhotra, P., and Srivastava, Y.V. (2020). Clustering Evaluation by Davies-Bouldin Index (DBI) in Cereal data using K-Means. *2020 Fourth International Conference on Computing Methodologies and Communication (ICCMC)*. doi:<https://doi.org/10.1109/iccmc48092.2020.iccmc-00057>.

Skillfloor (2023). *The Role of Data Preprocessing in Machine Learning*. [online] Medium. Available at: <https://skillfloor.medium.com/the-role-of-data-preprocessing-in-machine-learning-d9ad9db54d49>.

Smolic, H. (2024). *How to Use Machine Learning for Customer Segmentation*. [online] Medium. Available at: <https://hrvoje-smolic.medium.com/how-to-use-machine-learning-for-customer-segmentation-49612667301d> [Accessed 24 Jun. 2024].

Stedman, C. (2022). *What is Data Preparation? An In-Depth Guide to Data Prep*. [online] SearchBusinessAnalytics. Available at: <https://www.techtarget.com/searchbusinessanalytics/definition/data-preparation>.

Tavakoli, N., Siami-Namini, S., Adl Khanghah, M., Mirza Soltani, F. and Siami Namin, A. (2020). An autoencoder-based deep learning approach for clustering time series data. *SN Applied Sciences*, 2(5). doi:<https://doi.org/10.1007/s42452-020-2584-8>.

Tavakoli, N., Siami-Namini, S., Khanghah, M.A., Soltani, F.M. and Namin, A.S. (2020). *Clustering Time Series Data through Autoencoder-based Deep Learning Models*. [online] arXiv.org. doi:<https://doi.org/10.48550/arXiv.2004.07296>.

Tian, Y., Zhang, Y. and Zhang, H. (2023). Recent Advances in Stochastic Gradient Descent in Deep Learning. *Mathematics*, 11(3), p.682. doi:<https://doi.org/10.3390/math11030682>.

Tomar, V.S. (2022). A critical evaluation of activation functions for autoencoder neural networks - NORMA@NCI Library. *Ncirl.ie*. [online] doi:<https://norma.ncirl.ie/6329/1/vikalpsinghtomar.pdf>.

Turkmen, B. (2022). Customer Segmentation with Machine Learning for Online Retail Industry. *The European Journal of Social & Behavioural Sciences*, 31(2), pp.111–136. doi:<https://doi.org/10.15405/ejsbs.316>.

- Umuhuza, E., Ntirushwamaboko, D., Awuah, J. and Birir, B. (2020). Using Unsupervised Machine Learning Techniques for Behavioral-based Credit Card Users Segmentation in Africa. *SAIEE Africa Research Journal*, 111(3), pp.95–101. doi:<https://doi.org/10.23919/saiee.2020.9142602>.
- Varad R Thalkar (2021). Customer Segmentation Using Machine Learning. *International Journal of Scientific Research in Computer Science, Engineering and Information Technology*, pp.207–211. doi:<https://doi.org/10.32628/cseit217654>.
- Weerts, H.J.P., Mueller, A.C. and Vanschoren, J. (2020). Importance of Tuning Hyperparameters of Machine Learning Algorithms. arXiv:2007.07588 [cs, stat]. [online] Available at: <https://arxiv.org/abs/2007.07588>.
- Wu, H. and Wang, Q. (2022). GEAE: Gated Enhanced Autoencoder based Feature Extraction and Clustering for Customer Segmentation. *2022 International Joint Conference on Neural Networks (IJCNN)*. doi:<https://doi.org/10.1109/ijcnn55064.2022.9892475>.
- Xiao, J., Lu, J. and Li, X. (2017). Davies Bouldin Index based hierarchical initialization K-means. *Intelligent Data Analysis*, 21(6), pp.1327–1338. doi:<https://doi.org/10.3233/ida-163129>.
- Xu, Y., Wang, Q., An, Z., Wang, F., Zhang, L., Wu, Y., Dong, F., Qiu, C.-W., Liu, X., Qiu, J., Hua, K., Su, W., Xu, H., Han, Y., Cao, X., Liu, E., Fu, C., Yin, Z., Liu, M. and Roepman, R. (2021). Artificial Intelligence: a Powerful Paradigm for Scientific Research. *The Innovation*, [online] 2(4). doi:<https://doi.org/10.1016/j.xinn.2021.100179>.
- Yang, B., Fu, X., Sidiropoulos, N.D. & Hong, M... (2017). Towards K-means-friendly Spaces: Simultaneous Deep Learning and Clustering. Proceedings of the 34th International Conference on Machine Learning, in Proceedings of Machine Learning Research. Available from <https://proceedings.mlr.press/v70/yang17b.html>.
- Yang, Y., Zhang, K., and Kannan, P.K. (2021). EXPRESS: Identifying Market Structure: A Deep Network Representation Learning of Social Engagement. *Journal of Marketing*, 86(4), p.002224292110335. doi:<https://doi.org/10.1177/00222429211033585>.
- Yoseph, F., Ahamed Hassain Malim, N.H., Heikkilä, M., Brezulianu, A., Geman, O. and Paskhal Rostam, N.A. (2020). The impact of big data market segmentation using data mining and clustering techniques. *Journal of Intelligent & Fuzzy Systems*, 38(5), pp.6159–6173. doi:<https://doi.org/10.3233/jifs-179698>.

- Yoseph, F., Ahamed Hassain Malim, N.H., Heikkilä, M., Brezulianu, A., Geman, O. and Paskhal Rostam, N.A. (2020). The impact of big data market segmentation using data mining and clustering techniques. *Journal of Intelligent & Fuzzy Systems*, 38(5), pp.6159–6173. doi:<https://doi.org/10.3233/jifs-179698>.
- Yousefi, B., Khansari, M., Trask, R., Tallon, P., Carino, C., Afrasiyabi, A., Kundra, V., Ma, L., Ren, L., Farahani, K. and Hershman, M. (2024). *Density-based Isometric Mapping*. [online] arXiv.org. Available at: <https://arxiv.org/abs/2403.02531> [Accessed 5 Aug. 2024].
- Yuping, Z., Jílková, P., Guanyu, C. and Weisl, D. (2020). New Methods of Customer Segmentation and Individual Credit Evaluation Based on Machine Learning. *Proceedings of the 'New Silk Road: Business Cooperation and Prospective of Economic Development' (NSRBCPED 2019)*. doi:<https://doi.org/10.2991/aebmr.k.200324.170>.
- Zahra Shams Khoozani, Sabri, M., Woo Chaw Seng, Manjeevan Seera and Kah Yee Eg (2024). Navigating the landscape of concept-supported XAI: Challenges, innovations, and future directions. *Multimedia Tools and Applications*. doi:<https://doi.org/10.1007/s11042-023-17666-y>.
- Zare, H. and Emadi, S. (2020). Determination of Customer Satisfaction using Improved K-means algorithm. *Soft Computing*, 24(22), pp.16947–16965. doi:<https://doi.org/10.1007/s00500-020-04988-4>.
- Zhai, J., Zhang, S., Chen, J. and He, Q. (2018). Autoencoder and Its Various Variants. *2018 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*. doi:<https://doi.org/10.1109/smc.2018.00080>.
- Zhao, J. (2024). Customer segmentation application based on K-Means. *Applied and computational engineering*, 47(1), pp.242–247. doi:<https://doi.org/10.54254/2755-2721/47/20241400>.
- Zhou, J., Zhai, L. and Pantelous, A.A. (2020). Market segmentation using high-dimensional sparse consumers data. *Expert Systems with Applications*, [online] 145, p.113136. doi:<https://doi.org/10.1016/j.eswa.2019.113136>.

Appendix A: code

```
# Libraries & data Loading
```

```
## install libraries
# ! pip install yellowbrick
#Importing all required libraries
import pandas as pd
import numpy as np
np.random.seed(42)
import matplotlib.pyplot as plt
%matplotlib inline
from matplotlib import colors
from matplotlib.colors import ListedColormap
cmap = colors.ListedColormap(["#682F2F", "#9E726F", "#D6B2B1", "#B9C0C9",
"#9F8A78", "#F3AB60"])
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')
```

```
from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from yellowbrick.cluster import KElbowVisualizer
from sklearn.cluster import KMeans
```

```
from tensorflow.python.keras import Sequential
from tensorflow.keras import layers, optimizers
from tensorflow.keras.layers import *
from tensorflow.keras import Model
```

```
from sklearn.metrics import silhouette_score, calinski_harabasz_score
from tensorflow.keras.layers import Input, Dense, Lambda
from tensorflow.keras.losses import mse
from tensorflow.keras import backend as K
from sklearn.model_selection import train_test_split
import tensorflow as tf
from tensorflow.keras.optimizers import Adam
```

```
from tensorflow.keras.layers import Input, Dense, LeakyReLU, BatchNormalization, Dropout
```

```
from datetime import date
```

```
import pandas as pd
```

```

import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans, DBSCAN
from scipy.cluster.hierarchy import linkage, fcluster
from sklearn.metrics import pairwise_distances_argmin_min

#Reading the clustering_dfset
clustering_df = pd.read_csv('marketing_campaign.csv',sep='\t',parse_dates=['Dt_Customer'])
clustering_df.head()
#Information on features
clustering_df.info()
- There are missing values in income
- There are some categorical features in our dataset

# EDA
sns.countplot(clustering_df.Education).set(title = "Education level of Customer")
sns.countplot(clustering_df.Marital_Status).set(title = "Marital Status")
# Data Preprocessing

```

clustering_df preprocessing is a clustering_df mining technique which is used to transform the raw clustering_df in a useful and efficient format.

****Steps Involved in clustering_df Preprocessing:****

1. Data Cleaning
1. Data Transformation
1. Data Reduction

```

print(clustering_df.Education.value_counts())
#Segmenting Education levels in three groups
clustering_df.Education
clustering_df.Education.replace({'PhD':'PostGraduation','Master':'PostGraduation','2n
Cycle':'UnderGraduation','Basic':'UnderGraduation'})
print('-'*30)
print(clustering_df.Education.value_counts())
print(clustering_df.Marital_Status.value_counts())
# Segmenting Marital Status in two groups
clustering_df.Marital_Status
clustering_df.Marital_Status.replace({'Married':'Together','Single':'Alone','Divorced':'Alone','
Widow':'Alone','Absurd':'Alone','YOLO':'Alone'})
print('-'*40)
print(clustering_df.Marital_Status.value_counts())
# prompt: create box and whisker plot for df[inome]

```

```
import seaborn as sns
```

```

sns.boxplot(x=clustering_df['Income'])

clustering_df
# @title Data processing and feature Engineering
#Drop NA values
clustering_df.dropna(inplace=True)
#Dropping the outlier by setting a cap on income
clustering_df = clustering_df[clustering_df.Income<6000000]

#New feature for total spendings on various items and Time with the company
def get_age(birthyear):
    return date.today().year - birthyear
clustering_df['Age'] = clustering_df.Year_Birth.apply(get_age)

clustering_df["Spent"] = clustering_df["MntWines"] + clustering_df["MntFruits"] +\
clustering_df["MntMeatProducts"] + clustering_df["MntFishProducts"] +\
clustering_df["MntSweetProducts"] + clustering_df["MntGoldProds"]

clustering_df['Time_With_Company'] = pd.to_datetime(clustering_df.Dt_Customer, dayfirst
= True, format = '%d-%m-%Y')
clustering_df['Time_With_Company'] =
pd.to_numeric(clustering_df.Time_With_Company.dt.date\
               .apply(lambda z: (date.today() - z)).dt.days, downcast =
'integer') / 30

#New feature for total members in the household
clustering_df['Family_Size'] = clustering_df.Kidhome + clustering_df.Teenhome +
clustering_df.Marital_Status.replace({'Alone':1,'Together':2})

#Feature pertaining parenthood
clustering_df['Is_Parent'] = np.where((clustering_df.Kidhome +
clustering_df.Teenhome)>0,1,0)

#Dropping some of the features
clustering_df.drop(['ID','Year_Birth','Dt_Customer','Z_CostContact','Z_Revenue'],axis=1,inpl
ace=True)

clustering_df.head()

#Converting the labels into a numeric form so as to convert them into the machine-readable
form
label_encoder = LabelEncoder()
for item in ['Education','Marital_Status']:
    clustering_df[item] = clustering_df[[item]].apply(label_encoder.fit_transform)

```

```

clustering_df.head()
# Heat map among all numerical variables
cor = clustering_df.corr()
plt.figure(figsize = (27,26))
sns.heatmap(cor, annot = True, cmap = 'coolwarm')
plt.show()
#Standardizes a feature by subtracting the mean and then scaling to unit variance
scaler = StandardScaler()
scaled_clustering_df = scaler.fit_transform(clustering_df)
# scaled_clustering_df = scaled_clustering_df.copy()

# PCA

```

The higher the number of features, the harder it is to work with it. Many of these features are correlated, and hence redundant. This is why will be performing dimensionality reduction on the selected features before putting them through a classifier.

Dimensionality reduction is the process of reducing the number of random variables under consideration, by obtaining a set of principal variables (3 principal components).

```

#Transformation of clustering_df
pca = PCA(n_components=3)
pca_clustering_df = pd.DataFrame(pca.fit_transform(scaled_clustering_df),columns=['pca1',
'pca2', 'pca3'])
pca_clustering_df.head()
#Plotting the reduced clustering_df
ax = plt.axes(projection='3d')
for color in ['black', 'blue', '#B9C0C9']:
    n = 2340
    x, y = np.random.rand(2, n)
    scale = 200.0 * np.random.rand(n)

ax.scatter3D(pca_clustering_df['pca1'],pca_clustering_df['pca2'],pca_clustering_df['pca3'],c=
color,marker='o');
## Traditional Method of Clustering
# Define the range of clusters to try
# @title kmeans
def kmeans_clustering(scaled_clustering_df, k_range=range(2, 11)):
    for k in k_range:
        kmeans = KMeans(n_clusters=k, random_state=42)
        kmeans_labels = kmeans.fit_predict(scaled_clustering_df)
        silhouette_avg = silhouette_score(scaled_clustering_df, kmeans_labels)
        calinski_harabasz_avg = calinski_harabasz_score(scaled_clustering_df, kmeans_labels)
        print(f"K-means with {k} clusters")
        print(f"Silhouette Score: {silhouette_avg:.4f}")

```



```
print(f"Calinski-Harabasz Index: {calinski_harabasz_avg:.4f}\n")

# @title with pca K-means Clustering
kmeans_clustering(pca_clustering_df, k_range=range(2, 11))

# @title without pca K-means Clustering
kmeans_clustering(scaled_clustering_df, k_range=range(2, 11))
from sklearn.cluster import DBSCAN
from sklearn import metrics
import numpy as np
# @title DBSCAN
# Assume scaled_clustering_df is your scaled data
def dbscan_clustering(scaled_clustering_df, eps=8, min_samples=4):
    db = DBSCAN(eps=eps, min_samples=min_samples, metric='euclidean')
    model = db.fit(scaled_clustering_df)
    labels_ = model.labels_

    # Identify the core points
    sample_cores = np.zeros_like(labels, dtype=bool)
    sample_cores[db.core_sample_indices_] = True

    # Calculate the number of clusters
    n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
    print('Number of clusters:', n_clusters)

    # Calculate the silhouette score
    silhouette_score_ = silhouette_score(scaled_clustering_df, labels)
    print('Silhouette Score:', silhouette_score_)

    # Calculate the Calinski-Harabasz score
    calinski_harabasz_score_ = calinski_harabasz_score(scaled_clustering_df, labels)
    print('Calinski-Harabasz Score:', calinski_harabasz_score_)

# @title with pca DBSCAN Clustering
#dbscan_clustering(pca_clustering_df, eps=8, min_samples=4)

# @title without pca DBSCAN Clustering
dbscan_clustering(scaled_clustering_df, eps=8, min_samples=4)
# Define the range of clusters to try
# @title Hierarchical Clustering
def hierarchical_clustering(scaled_clustering_df, k_range=range(2, 11)):
    for k in k_range:
        linked = linkage(scaled_clustering_df, 'ward')
```

```

hierarchical_labels = fcluster(linked, k, criterion='maxclust')

silhouette_avg = silhouette_score(scaled_clustering_df, hierarchical_labels)
calinski_harabasz_avg = calinski_harabasz_score(scaled_clustering_df,
hierarchical_labels)
print(f"Hierarchical Clustering with {k} clusters")
print(f"Silhouette Score: {silhouette_avg:.4f}")
print(f"Calinski-Harabasz Index: {calinski_harabasz_avg:.4f}\n")

# @title without pca Hierarchical Clustering
hierarchical_clustering(pca_clustering_df, k_range=range(2, 11))
# @title without pca Hierarchical Clustering
hierarchical_clustering(scaled_clustering_df, k_range=range(2, 11))
# @title Autoencoder model
def build_autoencoder_hyper(input_dim, encoding_dim):
    input_layer = Input(shape=(input_dim,))

    # Encoder
    encoder = Dense(encoding_dim, activation=None)(input_layer)
    encoder = LeakyReLU(alpha=0.1)(encoder)
    encoder = BatchNormalization()(encoder)
    encoder = Dropout(0.2)(encoder)

    bottleneck = Dense(encoding_dim // 2, activation=None)(encoder)
    bottleneck = LeakyReLU(alpha=0.1)(bottleneck)
    bottleneck = BatchNormalization()(bottleneck)
    bottleneck = Dropout(0.2)(bottleneck)

    # Decoder
    decoder = Dense(encoding_dim, activation=None)(bottleneck)
    decoder = LeakyReLU(alpha=0.1)(decoder)
    decoder = BatchNormalization()(decoder)
    decoder = Dropout(0.2)(decoder)

    output_layer = Dense(input_dim, activation='sigmoid')(decoder)

    autoencoder = Model(inputs=input_layer, outputs=output_layer)
    autoencoder.compile(optimizer=Adam(learning_rate=0.001), loss='mse')

    # Encoder model to extract latent representations
    encoder_model = Model(inputs=input_layer, outputs=bottleneck)

    return autoencoder, encoder_model
def build_autoencoder(input_shape):
    """

```

Build and compile the autoencoder model.

Args:

input_shape (int): The number of input features.

Returns:

Model: Compiled autoencoder model.

Model: Encoder model.

"""

```
input_df = Input(shape=(input_shape,))
```

```
x = Dense(500, activation='relu')(input_df)
```

```
x = Dense(500, activation='relu')(x)
```

```
x = Dense(2000, activation='relu')(x)
```

```
encoded = Dense(2000, activation='relu')(x)
```

```
x = Dense(500, activation='relu')(encoded)
```

```
x = Dense(500, activation='relu')(x)
```

```
decoded = Dense(input_shape)(x)
```

```
autoencoder = Model(input_df, decoded)
```

```
autoencoder.compile(optimizer='adam', loss='mean_squared_error')
```

```
encoder = Model(input_df, encoded)
```

```
return autoencoder, encoder
```

```
# Variational Autoencoder model
```

```
# @title Variational Autoencoder (VAE) Model
```

```
def build_vae(input_dim, encoding_dim):
```

```
    input_layer = Input(shape=(input_dim,))
```

```
    h = Dense(500, activation='relu')(input_layer)
```

```
    z_mean = Dense(500 // 2)(h)
```

```
    z_log_var = Dense(500 // 2)(h)
```

```
def sampling(args):
```

```
    z_mean, z_log_var = args
```

```
    batch = tf.shape(z_mean)[0]
```

```
    dim = tf.shape(z_mean)[1]
```

```
    epsilon = tf.keras.backend.random_normal(shape=(batch, dim))
```

```
    return z_mean + tf.keras.backend.exp(0.5 * z_log_var) * epsilon
```

```
z = Lambda(sampling, output_shape=(500 // 2,))([z_mean, z_log_var])
```

```
decoder_h = Dense(500, activation='relu')
```

```
decoder_mean = Dense(input_dim, activation='sigmoid')
```

```

h_decoded = decoder_h(z)
x_decoded_mean = decoder_mean(h_decoded)

vae = Model(input_layer, x_decoded_mean)

def vae_loss(input_layer, x_decoded_mean):
    xent_loss = mse(input_layer, x_decoded_mean)
    kl_loss = -0.5 * K.sum(1 + z_log_var - K.square(z_mean) - K.exp(z_log_var), axis=-1)
    return xent_loss + kl_loss

vae.add_loss(vae_loss(input_layer, x_decoded_mean))
vae.compile(optimizer='adam')
return vae, Model(inputs=input_layer, outputs=z)

import tensorflow as tf
from tensorflow.keras.layers import Input, Dense, Lambda, LeakyReLU, BatchNormalization, Dropout
from tensorflow.keras.models import Model
from tensorflow.keras.losses import mse
from tensorflow.keras import backend as K

def build_vae_hyper(input_dim, encoding_dim):
    input_layer = Input(shape=(input_dim,))

    # Encoder
    h = Dense(encoding_dim, activation=None)(input_layer)
    h = LeakyReLU(alpha=0.1)(h)
    h = BatchNormalization()(h)
    h = Dropout(0.2)(h)

    z_mean = Dense(encoding_dim // 2)(h)
    z_log_var = Dense(encoding_dim // 2)(h)

    def sampling(args):
        z_mean, z_log_var = args
        batch = tf.shape(z_mean)[0]
        dim = tf.shape(z_mean)[1]
        epsilon = tf.keras.backend.random_normal(shape=(batch, dim))
        return z_mean + tf.keras.backend.exp(0.5 * z_log_var) * epsilon

    z = Lambda(sampling, output_shape=(encoding_dim // 2,))([z_mean, z_log_var])

    # Decoder
    decoder_h = Dense(encoding_dim, activation=None)
    decoder_h_output = decoder_h(z)

```

```

decoder_h_output = LeakyReLU(alpha=0.1)(decoder_h_output)
decoder_h_output = BatchNormalization()(decoder_h_output)
decoder_h_output = Dropout(0.2)(decoder_h_output)

decoder_mean = Dense(input_dim, activation='sigmoid')
x_decoded_mean = decoder_mean(decoder_h_output)

vae = Model(input_layer, x_decoded_mean)

def vae_loss(input_layer, x_decoded_mean):
    xent_loss = mse(input_layer, x_decoded_mean)
    kl_loss = -0.5 * K.sum(1 + z_log_var - K.square(z_mean) - K.exp(z_log_var), axis=-1)
    return K.mean(xent_loss + kl_loss)

vae.add_loss(vae_loss(input_layer, x_decoded_mean))
vae.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=0.001))

# Encoder model to extract latent representations
encoder_model = Model(inputs=input_layer, outputs=z_mean)

return vae, encoder_model

# Function to apply PCA
def apply_pca(data, n_components):
    pca = PCA(n_components=n_components)
    return pca.fit_transform(data)

# Function to perform clustering and evaluate
def evaluate_clustering(data, n_clusters=3):
    kmeans = KMeans(n_clusters=n_clusters, random_state=42)
    labels = kmeans.fit_predict(data)
    silhouette = silhouette_score(data, labels)
    calinski = calinski_harabasz_score(data, labels)
    return silhouette, calinski, labels

# Hyperparameter tuning function
def hyperparameter_tuning(train_data, test_data, data_scaled, model_type='autoencoder'):
    params = {
        'encoding_dim': [32, 64, 128],
        'batch_size': [128, 256],
        'epochs': [30, 50, 80]
    }

    results = []
    best_params = None
    best_score = -np.inf

```

```

for enc_dim in params['encoding_dim']:
    for batch in params['batch_size']:
        for epoch in params['epochs']:
            if model_type == 'autoencoder':
                model, encoder_model = build_autoencoder_hyper(train_data.shape[1], enc_dim)
            else:
                model, encoder_model = build_vae_hyper(train_data.shape[1], enc_dim)

            model.fit(train_data, train_data, epochs=epoch, batch_size=batch, shuffle=True,
validation_data=(test_data, test_data))
            encoded_data = encoder_model.predict(data_scaled)
            score = evaluate_clustering(encoded_data)[0] # Using silhouette score for simplicity

            results.append((enc_dim, batch, epoch, score))

            if score > best_score:
                best_score = score
                best_params = (enc_dim, batch, epoch)

return best_params, best_score, results
# Function to plot clusters
def plot_clusters(data, labels, title):
    plt.figure(figsize=(8, 6))
    unique_labels = np.unique(labels)
    for label in unique_labels:
        plt.scatter(data[labels == label, 0], data[labels == label, 1], label=f'Cluster {label}')
    plt.title(title)
    plt.legend()
    plt.show()
# Function to plot evaluation metrics
def plot_evaluation_metrics(results):
    methods = list(results.keys())
    silhouette_scores = [results[method][0] for method in methods]
    calinski_scores = [results[method][1] for method in methods]

    x = np.arange(len(methods))

    fig, ax1 = plt.subplots()

    ax1.set_xlabel('Method')
    ax1.set_ylabel('Silhouette Score', color='tab:blue')
    ax1.bar(x - 0.2, silhouette_scores, 0.4, label='Silhouette Score', color='tab:blue')
    ax1.tick_params(axis='y', labelcolor='tab:blue')

    ax2 = ax1.twinx()

```

```

ax2.set_ylabel('Calinski-Harabasz Index', color='tab:red')
ax2.bar(x + 0.2, calinski_scores, 0.4, label='Calinski-Harabasz Index', color='tab:red')
ax2.tick_params(axis='y', labelcolor='tab:red')

fig.tight_layout()
plt.xticks(x, methods)
plt.title('Clustering Evaluation Metrics')
plt.show()
# Function to plot hyperparameter tuning results
def plot_hyperparameter_tuning(results, model_type):
    encoding_dims, batch_sizes, epochs, scores = zip(*results)

    fig, ax = plt.subplots(figsize=(10, 6))
    sc = ax.scatter(encoding_dims, batch_sizes, c=scores, cmap='viridis', s=100)
    ax.set_xlabel('Encoding Dimensions')
    ax.set_ylabel('Batch Sizes')
    ax.set_title(f'Hyperparameter Tuning for {model_type.capitalize()} (Color: Silhouette Score)')
    plt.colorbar(sc, label='Silhouette Score')
    plt.show()

    fig, ax = plt.subplots(figsize=(10, 6))
    sc = ax.scatter(encoding_dims, epochs, c=scores, cmap='viridis', s=100)
    ax.set_xlabel('Encoding Dimensions')
    ax.set_ylabel('Epochs')
    ax.set_title(f'Hyperparameter Tuning for {model_type.capitalize()} (Color: Silhouette Score)')
    plt.colorbar(sc, label='Silhouette Score')
    plt.show()

from sklearn.model_selection import train_test_split
data_scaled = scaled_clustering_df
train_data, test_data = train_test_split(data_scaled, test_size=0.2, random_state=42)
input_dim = train_data.shape[1]
encoding_dim = 64
# Traditional Autoencoder
autoencoder, encoder_model = build_autoencoder(input_dim)
autoencoder.summary()
# Train the model and store the history
history_autoencoder = autoencoder.fit(train_data, train_data, epochs=100, batch_size=256,
shuffle=True, validation_data=(test_data, test_data))

# Plotting the training and validation loss
plt.figure(figsize=(10, 6))
plt.plot(history_autoencoder.history['loss'], label='Training Loss')
plt.plot(history_autoencoder.history['val_loss'], label='Validation Loss')

```

```
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.title('Training and Validation Loss Over Epochs Traditional Autoencoder')
plt.show()

# Encode the data
encoded_data = encoder_model.predict(data_scaled)
# autoencoder.fit(train_data, train_data, epochs=100, batch_size=256, shuffle=True,
validation_data=(test_data, test_data))
# encoded_data = encoder_model.predict(data_scaled)
# Variational Autoencoder
vae, vae_encoder_model = build_vae(input_dim, encoding_dim)
vae.summary()
history_vae= vae.fit(train_data, train_data, epochs=100, batch_size=256, shuffle=True,
validation_data=(test_data, test_data))

# Plotting the training and validation loss
plt.figure(figsize=(10, 6))
plt.plot(history_vae.history['loss'], label='Training Loss')
plt.plot(history_vae.history['val_loss'], label='Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.title('Training and Validation Loss Over Epochs Variational Autoencoder')
plt.show()

encoded_vae_data = vae_encoder_model.predict(data_scaled)
# Apply PCA
pca_data = apply_pca(data_scaled, n_components=2)
pca_encoded_data = apply_pca(encoded_data, n_components=2)
pca_encoded_vae_data = apply_pca(encoded_vae_data, n_components=2)
# pca_encoded_vae_data, encoded_vae_data, pca_encoded_data, encoded_data
#Elbow method to find numbers of clusters to make
elbow = KElbowVisualizer(KMeans(), k=10)
elbow.fit(pca_encoded_data)
elbow.show();
So the number of clusters is four (4)
# Evaluate clustering
results = {}
results['Autoencoder'] = evaluate_clustering(encoded_data, n_clusters=4)
results['Autoencoder PCA'] = evaluate_clustering(pca_encoded_data, n_clusters=4)
results['VAE'] = evaluate_clustering(encoded_vae_data, n_clusters=4)
results['VAE PCA'] = evaluate_clustering(pca_encoded_vae_data, n_clusters=4)
# Extracting the Silhouette Scores and Calinski-Harabasz Index
```



```

extracted_results = {method: (values[0], values[1]) for method, values in results.items()}

# Create a DataFrame from the results dictionary
df_results = pd.DataFrame.from_dict(extracted_results, orient='index', columns=['Silhouette Score', 'Calinski-Harabasz Index'])
df_results
results
results['Autoencoder'][2]
# Plot clusters
plot_clusters(pca_encoded_data, results['Autoencoder'][2], 'Autoencoder Clusters')
plot_clusters(pca_encoded_vae_data, results['VAE'][2], 'Variational Autoencoder Clusters')
plot_clusters(pca_data, results['Autoencoder PCA'][2], 'Autoencoder PCA Clusters')
plot_clusters(pca_encoded_vae_data, results['VAE PCA'][2], 'Variational Autoencoder PCA Clusters')
pca_clustering_df
kmeans = KMeans(4)
kmeans.fit(pca_encoded_data)
labels = kmeans.labels_
y = kmeans.fit_predict(pca_encoded_data)
pca_clustering_df['Cluster'] = y
#Adding the cluster to original clustering_dfset
clustering_df['Cluster'] = y
#3-D distribution of the clusters
plt.figure(figsize=(10,8))
ax = plt.axes(projection='3d')
ax.scatter3D(pca_clustering_df['pca1'],pca_clustering_df['pca2'],pca_clustering_df['pca3'],c=pca_clustering_df['Cluster'],cmap=cmap,marker='o',s=40);
# Plot evaluation metrics
plot_evaluation_metrics(results)
# @title Hyperparameter tuning
best_params_ae, best_score_ae, tuning_results_ae = hyperparameter_tuning(train_data, test_data, data_scaled, 'autoencoder')
best_params_vae, best_score_vae, tuning_results_vae = hyperparameter_tuning(train_data, test_data, data_scaled, 'vae')

print("Best Parameters for Autoencoder:", best_params_ae)
print("Best Silhouette Score for Autoencoder:", best_score_ae)
print("Best Parameters for VAE:", best_params_vae)
print("Best Silhouette Score for VAE:", best_score_vae)
# Traditional Autoencoder
autoencoder_hyper, encoder_model_hyper = build_autoencoder_hyper(input_dim, 32)
autoencoder_hyper.fit(train_data, train_data, epochs=50, batch_size=256, shuffle=True, validation_data=(test_data, test_data))
encoded_data_hyper = encoder_model_hyper.predict(data_scaled)
# Variational Autoencoder

```

```
vae_hyper, vae_encoder_model_hyper = build_vae_hyper(input_dim, 32)
vae_hyper.fit(train_data, train_data, epochs=80, batch_size=128, shuffle=True,
validation_data=(test_data, test_data))
encoded_vae_data_hyper = vae_encoder_model_hyper.predict(data_scaled)
# Apply PCA
pca_data_hyper = apply_pca(data_scaled, n_components=2)
pca_encoded_data_hyper = apply_pca(encoded_data_hyper, n_components=2)
pca_encoded_vae_data_hyper = apply_pca(encoded_vae_data_hyper, n_components=2)
# Evaluate clustering
results_hyper = {}
results_hyper['Autoencoder'] = evaluate_clustering(encoded_data_hyper)
results_hyper['Autoencoder PCA'] = evaluate_clustering(pca_encoded_data_hyper)
results_hyper['VAE'] = evaluate_clustering(encoded_vae_data_hyper)
results_hyper['VAE PCA'] = evaluate_clustering(pca_encoded_vae_data_hyper)

# Plot clusters
plot_clusters(pca_encoded_data_hyper, results['Autoencoder'][2], 'Autoencoder Clusters
Hyperparameter')
plot_clusters(pca_encoded_vae_data_hyper, results['VAE'][2], 'Variational Autoencoder
Clusters Hyperparameter')
plot_clusters(pca_data_hyper, results['Autoencoder PCA'][2], 'Autoencoder PCA Clusters
Hyperparameter')
plot_clusters(pca_encoded_vae_data_hyper, results['VAE PCA'][2], 'Variational Autoencoder
PCA Clusters Hyperparameter')
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd

# Updated data
data = {
    'Method': ['Autoencoder', 'Autoencoder PCA', 'VAE', 'VAE PCA', 'K-means', 'K-Means with
PCA', 'DBSCAN', 'DBSCAN PCA', 'Hierarchical', 'Hierarchical with PCA'],
    'Silhouette Score': [0.115328, 0.439293, 0.003647, 0.307383, 0.278314, 0.4268, 0.134897,
0.34334, 0.255103, 0.4253],
    'Calinski-Harabasz Index': [238.210063, 2299.436458, 8.008472, 1305.442805,
1123.890273, 1913.4799, 203.678912, 692.4556, 978.564782, 1680.6086]
}

# Create DataFrame
df = pd.DataFrame(data)

# Plotting
fig, ax1 = plt.subplots(figsize=(14, 8))

sns.set(style="whitegrid")
90
```

```
sns.set_palette("tab10")

# Silhouette Score barplot
sns.barplot(x='Method', y='Silhouette Score', data=df, ax=ax1)

# Create a second y-axis
ax2 = ax1.twinx()

# Calinski-Harabasz Index lineplot
sns.lineplot(x='Method', y='Calinski-Harabasz Index', data=df, ax=ax2, color='r', marker='o')

# Labels and titles
ax1.set_xlabel('Method', fontsize=14)
ax1.set_ylabel('Silhouette Score', fontsize=14, color='b')
ax2.set_ylabel('Calinski-Harabasz Index', fontsize=14, color='r')

ax1.set_xticklabels(df['Method'], rotation=45, ha='right')

plt.title('Comparison of Clustering Methods: Silhouette Score and Calinski-Harabasz Index',
          fontsize=16)
plt.show()

import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
from matplotlib.patches import Patch

# Updated data
data = {
    'Method': ['Autoencoder', 'Autoencoder PCA', 'VAE', 'VAE PCA', 'K-means', 'K-Means with
PCA', 'DBSCAN', 'DBSCAN PCA', 'Hierarchical', 'Hierarchical with PCA'],
    'Silhouette Score': [0.115328, 0.439293, 0.003647, 0.307383, 0.278314, 0.4268, 0.134897,
0.34334, 0.255103, 0.4253],
    'Calinski-Harabasz Index': [238.210063, 2299.436458, 8.008472, 1305.442805,
1123.890273, 1913.4799, 203.678912, 692.4556, 978.564782, 1680.6086]
}

# Create DataFrame
df = pd.DataFrame(data)

# Assign colors based on whether the method name contains 'PCA'
colors = ['b' if 'PCA' in method else 'g' for method in df['Method']]

# Plotting
fig, ax1 = plt.subplots(figsize=(15, 8))
91
```

```
sns.set(style="whitegrid")

# Silhouette Score barplot
bars = sns.barplot(x='Method', y='Silhouette Score', data=df, ax=ax1, palette=colors)

# Add different colors to bars
for bar, color in zip(bars.patches, colors):
    bar.set_color(color)

# Create a second y-axis
ax2 = ax1.twinx()

# Calinski-Harabasz Index lineplot
sns.lineplot(x='Method', y='Calinski-Harabasz Index', data=df, ax=ax2, color='r', marker='o')

# Labels and titles
ax1.set_xlabel('Method', fontsize=14)
ax1.set_ylabel('Silhouette Score', fontsize=14, color='b')
ax2.set_ylabel('Calinski-Harabasz Index', fontsize=14, color='r')

ax1.set_xticklabels(df['Method'], rotation=45, ha='right')

plt.title('Comparison of Clustering Methods: Silhouette Score and Calinski-Harabasz Index',
          fontsize=16)

# Add legend
legend_elements = [Patch(facecolor='b', label='With PCA'),
                   Patch(facecolor='g', label='Without PCA')]
ax1.legend(handles=legend_elements, loc='upper left')

plt.show()
```

Appendix B: Research Proposal



Project Title

An investigation into unsupervised machine Learning for predicting customer segment: Autoencoder method.

Project Overview

Aim: *This project aims to explore the application of unsupervised machine learning, specifically autoencoder, to predict customer segments based on their demographic and behavioural characteristics. The study will investigate the effectiveness of autoencoder over traditional methods in identifying distinct customer groups and providing insights for targeted marketing strategies*

Objectives:

- *Conduct a comprehensive literature review on unsupervised machine learning techniques for customer segmentation.*
- *To develop an autoencoder model for customer segmentation using demographic and behavioural data*
- *To evaluate the performance of the autoencoder model in identifying distinct customer segments.*
- *To compare the results of the autoencoder model with traditional clustering algorithms.*
- *To provide recommendations for businesses on the application of autoencoder for customer segmentation.*

Research Question: *How can autoencoder-based unsupervised machine learning be effectively utilized to identify and predict customer segments, and what advantages does it offer over traditional clustering methods?*

Practical outcomes:

- *A preprocessed and cleaned dataset suitable for training the autoencoder model on customer data.*
- *Development of a robust autoencoder model for customer segmentation*
- *An optimized autoencoder model architecture and hyperparameters for effective customer segmentation.*
- *A comparative analysis of the autoencoder's segmentation results against traditional clustering algorithms.*

Underpinning research with Literature Review

<i>Citation</i>	<i>Brief summary of paper</i>	<i>Relevance to your research question</i>	<i>Relevance to practical outcome of project</i>
Alkhayrat, M., Aljnidi, M. and Aljoumaa, K. (2020). A comparative dimensionality reduction study in telecom customer segmentation using deep learning and PCA. <i>Journal of Big Data</i> , 7(1). doi: https://doi.org/10.1186/s40537-020-0286-0 .	The paper proposes using an autoencoder neural network for dimensionality reduction on a large telecom customer dataset, before applying k-means clustering. It compares the autoencoder approach to PCA for dimensionality reduction. The autoencoder exhibited better clustering performance than PCA, suggesting its	The paper is relevant as it explores the application of autoencoder-based unsupervised learning for customer segmentation, highlighting its effectiveness in dealing with high-dimensional data and comparing it against traditional clustering methods such as PCA and k-means.	The study provides a comprehensive approach to preprocessing and cleaning datasets, developing a robust autoencoder model, and optimizing its architecture and hyperparameters. It also includes a comparative analysis of segmentation results from autoencoders and

	<i>effectiveness in extracting relevant features for customer segmentation</i>		<i>traditional methods, directly addressing the practical outcomes of data preparation, model development, and performance evaluation.</i>
Wu, H. and Wang, Q. (2022). GEAE: Gated Enhanced Autoencoder based Feature Extraction and Clustering for Customer Segmentation. 2022 <i>International Joint Conference on Neural Networks (IJCNN)</i> . doi: https://doi.org/10.1109/ijcnn55064.2022.9892475 .	<i>The paper introduces the Gated Enhanced Autoencoder (GEAE) model, which enhances clustering performance and reconstruction error compared to traditional methods. It highlights the importance of capturing nonlinear features in data and the impact of position selection of the gating block in improving model performance. The study focuses on customer segmentation using autoencoders for dimensionality reduction and</i>	<i>The paper addresses the research question by proposing the Gated Enhanced Autoencoder (GEAE) model for customer segmentation. It demonstrates how autoencoder-based unsupervised learning can efficiently reduce data dimensions and improve clustering accuracy by capturing complex features. This approach offers advantages over traditional clustering methods by enhancing feature extraction</i>	<i>Gives fundamental understanding of how to performed segmentation for customer using autoencoder alongside factors to consider in the data preprocessing and background understanding.</i>

	<i>emphasizes the significance of efficient feature extraction for clustering accuracy.</i>	<i>and clustering performance.</i>	
<i>Höchst, J., Lars Baumgärtner, Hollick, M. and Freisleben, B. (2017). Unsupervised Traffic Flow Classification Using a Neural Autoencoder. doi:https://doi.org/10.1109/lcn.2017.57.</i>	<i>The paper presents an unsupervised traffic flow classification approach using a neural autoencoder. It constructs feature vectors from statistical properties of network flows over exponentially growing time intervals. The autoencoder clusters the flows into different classes like downloads, uploads, calls, etc., without relying on predefined traffic classes. Evaluation on real data shows the approach can detect 7 different traffic classes with</i>	<i>The paper demonstrates how an autoencoder can be effectively used for unsupervised clustering and identification of different segments/classes in network traffic data, without relying on labelled data. This relates to using autoencoders for customer segmentation in an unsupervised manner.</i>	<i>The paper provides insights into preprocessing network traffic data into suitable feature vectors for the autoencoder model. It also highlights the importance of exploring different autoencoder architectures, hyperparameters, and scaling methods to optimize performance. The comparative analysis against traditional clustering methods like K-means can guide the evaluation of</i>

	<i>80% precision and 75% recall.</i>		<i>the autoencoder's segmentation results.</i>
<i>Mancisidor, R.A., Kampffmeyer, M., Aas, K. and Jenssen, R. (2021). Learning latent representations of bank customers with the Variational Autoencoder. Expert Systems with Applications, 164, p.114020. doi:https://doi.org/10.1016/j.eswa.2020.114020.</i>	<i>The paper explores the use of Variational Autoencoders (VAEs) for credit scoring and customer segmentation in the financial industry. It highlights the advantages of VAEs in capturing non-linear relationships and generating meaningful clusters in the latent space. The research demonstrates that segment-based credit scoring using VAEs outperforms traditional portfolio-based methods.</i>	<i>The paper is relevant as it highlights the effectiveness of autoencoder-based unsupervised learning in identifying and predicting customer segments, emphasizing the advantages over traditional clustering methods like improved performance and the ability to handle non-linear data relationships.</i>	<i>This research is directly applicable to the project outcomes as it provides a methodology for preprocessing customer data, developing and optimizing an autoencoder model, and conducting a comparative analysis with traditional clustering techniques. The findings support the practical implementation of VAEs for more accurate and insightful customer segmentation in a financial context.</i>

Nguyen, S.P. (2021). Deep customer segmentation with applications to a Vietnamese supermarkets' data. <i>Soft Computing</i>, 25(12), pp.7785–7793. doi:https://doi.org/10.1007/s00500-021-05796-0.	This paper focuses customer segmentation on Vietnamese supermarkets using soft computing and data mining techniques. The authors proposed a new method for customer segmentation that utilizes machine learning algorithms and data analysis to group customers into different segments based on their purchasing behaviour and other factors. The paper also discusses the potential benefits of this approach for personalized marketing and improving customer satisfaction.	The paper is highly relevant to the research question as it specifically addresses the utilization of autoencoder-based unsupervised machine learning for customer segmentation. It provides insights into the effectiveness of autoencoders in identifying and predicting customer segments and offers a comparative analysis against traditional clustering methods, thus helping to understand the advantages offered by this approach.	The preprocessing and cleaning techniques outlined in the paper will guide in preparing your dataset for model training. The development and optimization of the autoencoder model provides a framework that can be adapted to specific customer data.
--	--	---	---

Salehinejad, H. and Rahnamayan, S. (2016). Customer shopping pattern prediction: A recurrent neural network approach. <i>2016 IEEE Symposium Series on Computational Intelligence (SSCI)</i>. doi:https://doi.org/10.1109/ssci.2016.7849921.	<i>The paper proposes a recurrent neural network (RNN) model with an autoencoder component to predict customer shopping behaviour based on recency, frequency, and monetary (RFM) values. The autoencoder extracts features from the customer ID and RFM values, which are fed into the RNN to predict future RFM values. Experiments show the RNN with rectified linear units performed best for RFM prediction compared to simple RNNs and LSTMs.</i>	<i>The paper discusses the use of autoencoder-based unsupervised learning for customer segmentation, which aligns with your research question on identifying and predicting customer segments. It highlights how feature extraction through autoencoders can improve prediction accuracy compared to traditional methods.</i>	<i>Pre-processed and Cleaned Dataset: The paper provides insights into the preprocessing steps necessary for preparing transactional data for training the autoencoder model, including one-hot encoding for customer loyalty numbers.</i> <i>Development of a Robust Autoencoder Model: It details the architecture and training process of an autoencoder integrated within an RNN model, demonstrating its application in customer segmentation.</i> <i>Optimized Model Architecture and Hyperparameters : The paper outlines the specific architecture and hyperparameters used, such as the number of hidden units, epochs, and autoencoder features, which can guide the</i>
---	--	--	---

			optimization of your model.
Soni, J., Prabakar, N., Upadhyay, H. (2023). <i>Deep Learning-Based Efficient Customer Segmentation for Online Retail Businesses</i>. In: Yang, XS. (eds) <i>Benchmarks and Hybrid Algorithms in Optimization and Applications</i>. Springer Tracts in Nature-Inspired Computing. Springer, Singapore. https://doi.org/10.1007/978-981-99-3970-1_9	The paper discusses the importance of customer segmentation in today's competitive market and proposes a learning-based framework using unsupervised machine learning algorithms, specifically K-Means and Autoencoders, to segment customers. The framework is implemented on an open-source dataset from Kaggle, and the results are compared with traditional clustering methods. The paper aims to provide an effective	The paper directly addresses the research question by exploring the effectiveness of AutoEncoder-based unsupervised machine learning in identifying and predicting customer segments. It provides a comprehensive analysis of the advantages of Autoencoders over traditional clustering methods, thereby answering the research question.	The paper aligns with the practical outcomes by providing a preprocessed and cleaned customer dataset, developing a robust autoencoder model for segmentation, optimizing the model architecture and hyperparameters, and conducting a comparative analysis of the autoencoder's segmentation results against traditional clustering algorithms. These outcomes can directly contribute to the

	<i>way to identify and predict customer segments using Autoencoder-based unsupervised machine learning.</i>		<i>practical implementation of customer segmentation using deep learning techniques.</i>
<i>Smutek, T., Sikora, J., Bogacki, S., Rutkowski, M. and Woźniak, D., 2024, Use of Autoencoder and One-Hot Encoding for Customer Segmentation.</i>	<i>The research paper applies Autoencoder and One-Hot Encoding techniques to segment retail customers based on their profiles. It evaluates the effectiveness of these methods using various clustering algorithms like K-means, DBSCAN, Louvain Community Detection, Greedy Modularity, and Label Propagation. The study finds that the Louvain method with AutoEmbedder produces more cohesive and meaningful customer segments</i>	<i>The paper directly addresses the utilization of autoencoder-based unsupervised machine learning for customer segmentation. It demonstrates the advantages of autoencoders over traditional One-Hot Encoding by forming more modular and cohesive customer clusters. The comparative analysis with established clustering algorithms highlights the potential benefits of using autoencoders for this task.</i>	<i>The research provides a practical approach to preprocess customer data and train an autoencoder model for segmentation. It offers insights into optimizing the autoencoder architecture and evaluating its performance against benchmarks. The clustering quality metrics used can guide the selection of appropriate algorithms and parameters for effective customer segmentation in</i>

	<i>compared to One-Hot Encoding, as evidenced by higher modularity scores.</i>		<i>practical applications.</i>
Tavakoli, N., Siامي-Namini, S., Khanghah, M.A., Soltani, F.M. and Namin, A.S. (2020). <i>Clustering Time Series Data through Autoencoder-based Deep Learning Models</i>. [online] arXiv.org. doi:https://doi.org/10.48550/arXiv.2004.07296.	<i>The paper introduces an autoencoder-based synergic methodology for time series clustering, specifically focusing on financial data. It details the architecture of an encoder-decoder neural network designed to learn features from time series data, emphasizing the ability to capture salient features and reconstruct original inputs for clustering.</i>	<i>The paper is directly relevant as it explores how autoencoder-based unsupervised learning can effectively cluster time series data, demonstrating the advantages of capturing hidden features over traditional clustering methods.</i>	<i>This paper provides detailed insights into the development and optimization of an autoencoder model, including architecture and hyperparameters, for clustering tasks. The comparative analysis with traditional clustering methods aligns with your project's goals of evaluating the efficacy of autoencoder-based segmentation.</i>

Wang, C. (2022). Efficient customer segmentation in digital marketing using deep learning with swarm intelligence approach. <i>Information Processing & Management</i> , 59(6), p.103085. doi: https://doi.org/10.1016/j.ipm.2022.103085 .	<i>The paper proposes an unsupervised deep learning model that combines a self-organizing map (SOM) with a modified social spider optimization (MSSO) algorithm for customer segmentation in digital marketing. Features are selected using MSSO, and the SOM clusters customers based on their purchasing patterns. The clustered data is then classified using a deep neural network (DNN) to segment customers into different groups for targeted marketing strategies.</i>	The paper is highly relevant as it explores the use of an autoencoder-based unsupervised deep learning model (SOM) for customer segmentation. It demonstrates how the autoencoder can effectively identify and group customers based on their purchasing behaviour, which is a key aspect of your research question on utilizing autoencoders for customer segmentation.	<i>The paper provides a practical approach to preprocess and clean the customer dataset using feature selection techniques like MSSO. It also outlines the development of the SOM autoencoder model for segmentation and optimizes its architecture through quantization and topological error analysis. Additionally, it compares the autoencoder's performance with traditional clustering methods, fulfilling the practical outcomes outlined in your project.</i>
--	--	--	---

Project Schedule

ask Id	Task Name	Start	Deadline	Hour	Deliverable
1.0	Project initiation and Literature Review				
1.1	Project aim and objective declaration	09/06/2024	09/06/2024	6	Draft of chapter one for the

					customer segmentation
1.1	Identify relevant publications	10/06/2024	10/06/2024	4	List of relevant publications on customer segmentation, traditional cluster model and autoencoder
1.2	Analyze and synthesize findings	11/06/2024	11/06/2024	7	Summary of key findings on the selected areas of discussions
1.3	Write literature review report	12/06/2024	13/06/2024	15	Literature review report on segmentation practices, autoencoder
1.4	Completion of chapter 1	14/06/2024	15/06/2024	15	Complete report of chapter 1
2.0	Dataset Preparation				
2.1	Sourcing for dataset	16/06/2024	16/06/2024	6	List of identified datasets to use from UCI or Kaggle
2.2	Collection of open-source customer dataset	17/06/2024	18/06/2024	10	Selected raw dataset from kaggle
2.3	Preprocessing of collected dataset	19/06/2024	20/06/2024	12	Preprocessed and annotated dataset
2.4	Documentation of chapter two	21/06/2024	22/06/2024	15	Complete report of chapter 2
3.0	Model Development				
3.1	Identification and selection of research methodology	23/06/2024	24/06/2024	2	Summary of the research methodology
3.2	Exploration autoencoders for segmentation	25/06/2024	25/06/2024	3	List of combination of different uses of autoencoder for segmentation

3.3	Python code implementation of the selected algorithm	26/06/2024	28/06/2024	15	Code artifact of the implementation
3.4	Training of the autoencoder model	29/06/2024	03/07/2024	23	Trained deep learning models
3.4	Testing of trained model	04/07/2024	04/07/2024	2	Performance report of model
3.5	Refine and optimize models	05/07/2024	05/07/2024	5	Refined and optimized models
3.4	Documentation of chapter 3	06/07/2024	09/07/2024	15	Complete report of chapter 3
4.0	Evaluation and Benchmarking				
4.1	Gathering of research result	10/07/2024	10/07/2024	4	First stage summary of the research result
4.1	Define evaluation metrics	11/07/2024	12/07/2024	1.5	Evaluation metrics and benchmarks
4.2	Evaluate model performance	13/07/2024	14/07/2024	4	Performance evaluation results
4.3	Benchmark against other methods	15/07/2024	16/07/2024	3	Benchmarking report
5.0	Evaluation and Reporting				
5.1	Completion of chapter 4	17/07/2024	19/07/2024	15	Complete report of chapter 4
5.2	Identification if objectives have been met	20/07/2024	22/07/2024	9	Refined and achievement table
6.0	Documentation				
6.1	Conclusion documentation	23/07/2024	25/07/2024	15	Complete report on chapter 5
6.2	Finalize project documentation	26/07/2024	04/08/2024	25	Completed project documentation
6.3	Preparation for Viva	05/08/2024	06/08/2024	3	Complete PowerPoint presentation
6.4	Submissions of report and artifacts	10/08/2024	10/08/2024	1	Submitted report and artifacts

Deliverables

Del. No.	Name	Deadline
D1	Complete report of chapter 1	15/06/2024
D2	Preprocessed and annotated dataset	20/06/2024
D3	Complete report of chapter 2	22/06/2024
D4	Summary of the research methodology and exploration of autoencoders for segmentation	25/06/2024
D5	Code artifact and trained deep learning models	03/07/2024
D6	Performance report and refined/optimized models	05/07/2024
D7	Complete report of chapter 3	09/07/2024
D8	Performance evaluation results and benchmarking report	16/07/2024
D9	Complete report of chapter 4	19/07/2024
D10	Finalized project documentation and complete PowerPoint presentation	06/08/2024
D11	Submitted report and artifacts	10/08/2024

Milestones

Milestone	Name	Deadline	Evidence
M1	Project Initiation	09/06/2024	Project proposal, project plan
M2	Literature Review Completion	15/06/2024	Literature review report, chapter 1
M3	Dataset Preparation Completion	29/06/2024	Dataset preparation report, chapter 2
M4	Model Development Completion	16/07/2024	Model development report, chapter 3
M5	Evaluation and Benchmarking Completion	26/07/2024	Evaluation and benchmarking report, chapter 4
M6	Final Reporting Completion	01/08/2024	Final report, chapter 5
M7	Project Documentation Completion	05/08/2024	Completed project documentation

M8	Final Submission	10/08/2024	Submitted report and artifact
----	------------------	------------	-------------------------------

Outline Schedule / Gantt chart

	June					July					August	
	1	2	3	4	5	6	7	8	9	10	11	12
Project Initiation	M1 D1											
Literature Review			M2									
Dataset Preparation				M3 D2								
Model Development						D4	M4 D5					
Evaluation and Benchmarking									M5			
Final Reporting										D9 M6		
Project Documentation											M7	
Final Submission												M8 D11

Evaluation Plan

Objective	Evaluation Approach	Evidence
Conduct a comprehensive literature review on unsupervised machine learning techniques for customer segmentation.	Systematic review of relevant publications, categorizing and synthesizing findings to identify gaps and opportunities for research contribution.	D1 and D2: List of relevant publications; Comprehensive literature review report. The list of relevant publications indicates thorough research into existing approaches, challenges, and gaps in the field. Additionally, the comprehensive literature review reports the synthesis of findings.
To develop an autoencoder model for customer segmentation using	Design and implement an autoencoder model, preprocess demographic and behavioral data, train and optimize the model	D3: Preprocessed and annotated dataset; D4: Code artifact and trained deep learning models. The preprocessed dataset ensures data quality, while the code artifact and

demographic and behavioral data.	for effective segmentation.	trained models demonstrate the practical application and development of the autoencoder.
To evaluate the performance of the autoencoder model in identifying distinct customer segments.	Use performance metrics such as accuracy, precision, recall, F1 score, and visualization techniques to assess the effectiveness of the model.	D5: Performance report and refined models. The performance report provides quantitative and qualitative assessments of the model's accuracy and effectiveness in segmenting customers.
To compare the results of the autoencoder model with traditional clustering algorithms.	Implement traditional clustering algorithms (e.g., K-means, hierarchical clustering) and compare their results with those of the autoencoder model.	D6: Evaluation results and benchmarking report. This report includes comparative analyses highlighting the strengths and weaknesses of the autoencoder model relative to traditional methods.
To provide recommendations for businesses on the application of autoencoder for customer segmentation.	Synthesize findings from the literature review, model development, and performance evaluation to formulate actionable business recommendations.	D7: Finalized project documentation and presentation. The documentation includes detailed recommendations, while the presentation highlights key insights for business stakeholders.

Social, Ethical, Legal and Professional issues

Social issues	Bias and discrimination in customer segmentation, privacy concerns with collecting and using customer data which will be mitigated by using open-source data. Additionally, they may be unintended consequences or misuse of customer segmentation models.
Ethical issues	Potential for perpetuating historical biases or unfair treatment, balancing commercial interests with ethical considerations and ensuring algorithmic oversight and alignment with societal values.
Professional issues	Ensuring proper training and expertise in developing and deploying AI systems. Maintaining professional standards and ethical practices in AI development. Responsible AI practices, including bias testing, model auditing, and stakeholder engagement.
Legal Issues	Compliance with data privacy regulations (e.g., GDPR, CCPA), Furthermore, fairness and non-discrimination laws

in customer segmentation and targeting. Ensuring Intellectual property rights and ownership of AI models, customer data and research papers utilized.